claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-aeebed079c3f88579.jsonl`
- SHA-256: `05838e1cc05372949fca11100bf24061dc5f4ff6e4a40ad690a6b878fcfbaf1a`
- Source modified: `2026-06-14T10:56:05+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T10:41:35.754Z)

Pick up Personalization Phase-1, the FIRST SAFE SLICE only (serving-bridge groundwork ? additive, ZERO served-behavior change). Repo cwd C:/Users/avidu/Projects/badminton-highlight-indexer. ISOLATED worktree: `git -C ... fetch origin` then `git -C ... worktree add C:/Users/avidu/Projects/bhi-servingbridge -b feat/personalization-store-and-bridge origin/master`.

Read docs/PERSONALIZATION_PLAN.md (§2 the per-user data model), backend/infrastructure/database.py (the `PRAGMA user_version` migration ladder, current=v3), backend/eval/classifier.py (LogisticRegression save/load/from_dict), backend/pipeline/segmenters/fusion_hybrid.py (`_select_for_handoff` ? the FUTURE wiring point; DO NOT change it in this PR).

BUILD (additive; NULL user_id = local = byte-identical to today; NO served-behavior change):
- Migration **v4** (nullable `user_id` on `videos` + `candidate_segments` + indexes) and **v5** (`user_models` store per PLAN §2: user_id, version, baseline_version, feature_schema_hash, fusion_config JSON, classifier_json, promotion_evidence, n_videos_at_fit, is_active) following the existing migration + test pattern; tests assert the version bump + NULL-user back-compat.
- `backend/pipeline/personalization/serving.py`: `resolve_user_fusion_config(db, user_id, served_cfg)` (overlay the active user_models row onto the served fusion cfg; fall back to baseline if no active model or n_videos_at_fit < min_n) + `load_user_classifier(db, user_id)` (LogisticRegression.from_dict on the stored classifier_json). NOT yet wired into `_select_for_handoff` ? leave a clear TODO + docstring noting that served wiring is the NEXT owner-gated PR.
- CRUD helpers (`upsert_user_model` / `get_active_user_model`) + tests.

COMPREHENSIVE tests (healthy coverage). ruff + mypy + full pytest green. Self-MERGE on green (additive/safe, within the merge agreement): gh pr create --base master, merge --squash --delete-branch, remove worktree + branch + prune. Branch-safe (explicit paths; `git log origin/master..HEAD` = only your commit). `Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>`.

Return: PR number/URL, what was built, test results, merge status, and the EXACT description of the NEXT PR (the served-wiring into `_select_for_handoff`, owner-gated) for the handoff.

2. assistant (2026-06-14T10:41:38.437Z)

I'll start by ramping up on the relevant docs and code, then build the additive slice in an isolated worktree.

3. user (2026-06-14T10:41:41.432Z)

(Bash completed with no output)

4. user (2026-06-14T10:41:41.441Z)

Personalization Plan ? a per-user feature on a generalization-first baseline

Status: DESIGN ? owner-adopted (hybrid); Phase-0 engineering landing. This is the authoritative plan for the personalization track. The detailed strategy analysis + the full pros/cons that led here live in the working artifacts `scratch/PERSONALIZATION_PIVOT_ANALYSIS.md` (the decision-grade counter-case) and `scratch/PERSONALIZATION_FEATURE_DESIGN.md` (the code-grounded design); this doc is the checked-in summary + decisions + phased build plan. Peer to OWNED_MODEL_IMPLEMENTATION_PLAN.md, EXPERIMENT_HARNESS.md, and MULTI_SIGNAL_FUSION_PLAN.md.

0. Thesis & strategic rationale (why *hybrid*, not personalization-first)

Foundation principle (owner): *"a tool that gets better the more you help it."* Contribution (data / video / diagnostics / annotation) improves BOTH the user's own model AND the shared baseline; **payoff scales with contribution.**

A directional pivot ? *personalization-first* (ship a baseline that continually self-optimizes to one user's own footage) ? was researched in depth. The verdict was **HYBRID, not all-in**:

- **Adopt personalization as a flag-gated FEATURE on a generalization-first baseline.** The harness genuinely makes per-user *tuning* near-free (it is already a corpus-relative A/B engine; a per-user corpus is just a special case). This is real and worth shipping.
- **Do NOT bet the strategy on per-user *learning*.** Per-user A/B is the n?6 small-sample regime *worse* (sharding data when you should pool); the differentiated parts (a learned per-user model serving path, the correction?label write path, a user/tenant dimension) are largely unbuilt; a naively-overfit per-user model loses the cross-user data flywheel and has no shared baseline to regress against.
- **The owner's contribution-flywheel + free-tier-pooling closes the one real hole** the counter-case found: by having (free-tier) users contribute to the *shared* baseline, the compounding cross-user moat is preserved *alongside* personalization ? personalization **and** flywheel, not either/or.

Net: a generalization-first **baseline** ("batteries"), a thin **per-user tuning** layer gated by the honest small-N statistics (with a hard floor + structural-guard protection), and a **contribution loop** that feeds both.

1. Owner decisions (locked 2026-06-14)

1. **Identity** ? auth WILL land; *design for the north star (multi-user / cloud), implement for current use (single-user local)*. `user_id` nullable now (NULL = local install), device/install UUID for the alpha; real auth + a `users` table later.
2. **Per-user model storage** ? inline `classifier_json` in the (local) DB now, but **behind a load interface** so that post-auth it is fetched from the **cloud-hosted DB run by the service** that

```
43         generates base models and orchestrates the product. The local store is one
implementation of that
44         interface.
45      3. **`min_n` is a PROMOTION-confidence floor, NOT a service/segmentation gate.** A
brand-new user with
46         one video is ALWAYS fully segmented by the baseline batteries ? never denied, never
"give me more
47         videos first." `min_n` only gates whether a *per-user* A/B is trusted enough to
OVERRIDE a baseline
48         cue default *for that user*; below it (or if the CI / sign test don't clear) ? keep
the baseline.
49         **Training-corpus policy:** *no more than needed, no less than all-if-few* ? fit on
the
50         `smallest_k_with_lift` videos (the held-out-USER learning curve), else ALL available.
51      4. **"Flip Gate-A to default"** = ship the validated Gate-A held-shuttle guard as the
**default
52         detection path** (today it lives in the opt-in `FusionSegmenter`, which is not the
default
53         segmenter). Changes served behavior ? **gated on a golden-set regression run + owner
OK**; still
54         open (see §5).
55      5. **Flywheel = backend property** (not in-product marketing). A **free tier** is likely
and
56         **free-tier videos ARE pooled** into the baseline (consent baked into the free terms;
57         minors-exclusion still enforced) ? the clean value-exchange that feeds the owner-
trained base model.
58         Paid tiers get stronger privacy / opt-out.
59
60      **Pending sub-decision:** structural-guard scope ? recommended **Gate-A only** (the
proven structural
61      held-shuttle guard; its value is in failure modes a user's corpus may not yet contain),
with **Gate-B**
62      (person-occupancy / `min_players`) left *subject* to per-user A/B; **`min_n = 5`**.
**Gate-A-only
63      structural, `min_n = 5` ? owner-confirmed 2026-06-14.**
64
65      ## 2. Architecture
66
67      **Two-layer decoupling (already real).** Detection (GPU, run once): `FusionSegmenter`
persists the
68      **full superset** of per-window features into `candidate_segments.stats`; only the
served handoff is
69      gated. Decision (CPU, swappable): `experiment.Variant` + `predict()` re-score persisted
features.
70      **Personalization == a per-user `Variant`** (cue_flags + `min_crossings`/`min_players` +
optional
71      LogReg + threshold). No segmenter rewrite ? a per-user `FusionConfig` overlay resolved
upstream.
72
73      **The single highest-leverage wiring:** `fusion_hybrid.py::_select_for_handoff` is the
production
74      decision seam. When a per-user model is pinned AND the promotion gate says ON, apply
75      `LogisticRegression.predict_proba` over `window_stats` (the same math
`experiment.predict()` runs
76      offline). `LogisticRegression.load` is **never called in `backend/pipeline/` today** ?
the serving
77      bridge is the #1 wall.
78
79      **Per-user data model** (migrations follow the `PRAGMA user_version` ladder; current =
v3):
80      - **v4** ? nullable `user_id` on `videos` + `candidate_segments` (NULL = local; byte-
identical to today).
81      - **v5** ? `user_models` store: `(user_id, version, baseline_version,
feature_schema_hash,
82        fusion_config, classifier_json, promotion_evidence, n_videos_at_fit, is_active)`.
```

Inline
83   `classifier_json` (tenant-safe); `feature_schema_hash` is the MINOR-vs-MAJOR upgrade
switch.
84   - **v6** ? `contribution_consent` ledger (default-deny; minors-exclusion enforced **at
the pooling
85   query**, not advisory).
86
87   ## 3. Constructs (the gap list + what is built)
88
89 | # | Construct | Status |
90 |---|---|---|
91 | 1 | Per-user store + serving bridge (wire `LogisticRegression.load` into
`_select_for_handoff`) | Phase 1 (the #1 wall) |
92 | 2 | Correction-loop ? per-user labels (`human_label` setter + confirm/reject API;
recall-protecting de-bias prior) | Phase 2 (`human_label` is a read-only stub today) |
93 | 3 | **Per-user promotion gate** (min-N floor + structural-guard exemption) | ? **DONE
? PR #141** (`backend/eval/per_user_gate.py`) |
94 | 4 | Contribution flywheel (opt-in data/video/diagnostics ? shared baseline; payoff ?
contribution; consent + minors-exclusion) | Phase 2 |
95 | 5 | User-as-annotator charter (rally-annotator: auto-generate a ~2-min high-
uncertainty segment from the user's own video; guide labeling ? grounding) | Phase 3 |
96 | 6a | **Held-out-USER learning curve** (the falsifiable "<K videos to superior quality"
criterion) | ? **DONE ? PR #142** (`backend/eval/per_user_eval.py`) |
97 | 6b | Per-user no-regression guard (frozen per-user CONTROL; never accept a regressing
retrain/upgrade) | ? **in flight** (`feat/per-user-no-regression`) |
98 | 7 | Day-0 "batteries" = the validated label-free Gate-A/B fusion (?F1 +0.074, 6/6,
p=0.031) shipped as the default | Phase 0, owner-gated (§5) |
99
100   ## 4. Phased PR-sized build plan + status
101
102   **Phase 0 ? safety + day-0 batteries** (no schema / API / served-behavior change; pure-
Python + tests):
103   - PR-0.1 per-user promotion gate ? ? **#141 merged**
104   - PR-0.2 held-out-USER learning curve ? ? **#142 merged**
105   - PR-0.3 per-user no-regression guard ? ? in flight
106   - PR-0.4 ship Gate-A as the day-0 battery ? **owner-gated** (§5; needs a golden-set
regression run)
107   - PR-0.5 within-user LOVO measurement (the "confirm-or-kill" experiment) ? **owner-
gated** (needs multi-video-per-user data on the GPU box)
108
109   **Phase 1 ? per-user tuning serving + store:** v4 migration +
`set_human_label`/`set_user_id`; v5
110   `user_models` + CRUD; `serving.py` overlay resolver + `load_user_classifier`; **wire the
bridge into
111   `_select_for_handoff` (owner-gated ? served path)**; `/retune` + `/model` trust-feature
endpoints.
112
113   **Phase 2 ? correction loop + flywheel:** `POST /api/candidates/{id}/label` (the first
`human_label`
114   writer); recall-protecting de-bias prior; v6 consent ledger + minors-exclusion; **opt-in
contribution
115   upload (owner-gated ? privacy surface)**; free-tier pooling.
116
117   **Phase 3 ? annotator charter + taxonomy + baseline upgrades:** active-learning task
picker; rally-
118   annotator CSV ingest ? per-user labels; **coarse hard/soft shot proxy (owner-gated)**;
**baseline-
119   upgrade governance (MINOR carry / MAJOR re-fit on `feature_schema_hash`) + optional
federated nudge
120   (owner-gated, counsel).**
121
122   ## 5. Open decisions (owner)
123
124   1. **Structural-guard scope + `min_n` value** ? lock **Gate-A-only structural, `min_n =
5`** (recommended)?

```
125     2. **Gate-A default flip (PR-0.4)** ? Gate-A lives in the non-default `FusionSegmenter`,
so "ship as
126        default" is a path choice (make that segmenter the default vs port the gate into the
default path).
127        Recommended sequence: **run a golden-set regression first**, bring the number, *then*
approve.
128     3. **Per-user `user_id` for the alpha** ? device/install UUID acceptable, or must auth
land first?
129     4. **Local-install flywheel default** ? fully-local zero-pooling until consent is
explicitly granted?
130
131     ## 6. Honest risks (carried from the analysis)
132
133     - **Small-N per-user eval noise** is central and *worse* per-user ? mitigated
structurally: the gate
134       refuses below `min_n` and falls back to the baseline; never auto-promotes in the
regime the tooling
135       exists to refuse in. PR-0.5 confirms-or-kills the core premise before any serving
build-out.
136     - **Cold-start** ? the day-0 Gate-A battery (zero per-user labels) is the floor for the
median user.
137     - **Privacy deferral** ? do not advertise "data never leaves your machine" until on-
device ships;
138       near-term pooling carries the full cloud (DPA / consent / minors) posture.
139     - **Compute** ? heavyweight VLM personalization (on-device Qwen3-VL fine-tune) is **out
of v1 scope**;
140       v1 is the cheap LogReg / threshold / cue-ON-OFF layer only.
141
142     ## 7. References
143     - `scratch/PERSONALIZATION_PIVOT_ANALYSIS.md` ? the decision-grade strategy analysis
(pros/cons, harness-fit, counter-case, recommendation).
144     - `scratch/PERSONALIZATION_FEATURE_DESIGN.md` ? the code-grounded design (schema,
serving bridge, APIs, critical files).
145     - Built: `backend/eval/per_user_gate.py` (#141), `backend/eval/per_user_eval.py` (#142).
146     - Harness: [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md), `backend/eval/experiment.py`,
`backend/eval/eval_stats.py`.
147
```

5. assistant (2026-06-14T10:41:44.561Z)

```
I have the plan. Now let me read the key code files and set up the isolated worktree in
parallel.
```

6. user (2026-06-14T10:41:46.398Z)

```
Preparing worktree (new branch 'feat/personalization-store-and-bridge')
branch 'feat/personalization-store-and-bridge' set up to track 'origin/master'.
HEAD is now at e75ec35 feat(eval): LOSO/LOGO ablation runner ? per-signal marginal ?F1 (Phase 0)
(#147)
```

7. user (2026-06-14T10:41:46.891Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
```

```
13
14          @contextmanager
15          def _connect(self):
16              """Transaction scope that also CLOSES the connection.
17
18              ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19              never closes, so every call leaked its connection to GC (lingering file
20              handles on the WAL sidecars under Windows). Internal methods use this.
21              """
22              conn = self._get_connection()
23              try:
24                  with conn:
25                      yield conn
26              finally:
27                  conn.close()
28
29          def _get_connection(self):
30              # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31              # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
32              conn = sqlite3.connect(self.db_path, timeout=30.0)
33              conn.row_factory = sqlite3.Row
34              # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
35              conn.execute("PRAGMA foreign_keys = ON")
36              # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
37              conn.execute("PRAGMA busy_timeout = 30000")
38              try:
39                  conn.execute("PRAGMA journal_mode = WAL")
40              except sqlite3.OperationalError:
41                  pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
42              return conn
43
44          def _init_db(self):
45              """Initializes tables for videos, play_segments, and events."""
46              with self._connect() as conn:
47                  cursor = conn.cursor()
48
49                  # Videos Table
50                  cursor.execute("""
51                      CREATE TABLE IF NOT EXISTS videos (
52                          id TEXT PRIMARY KEY,
53                          filepath TEXT NOT NULL,
54                          processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
55                          status TEXT NOT NULL,
56                          validation_results TEXT
57                      )
58                  """)
59
60                  # Play Segments Table (Extensible metadata JSON)
61                  cursor.execute("""
62                      CREATE TABLE IF NOT EXISTS play_segments (
63                          id INTEGER PRIMARY KEY AUTOINCREMENT,
64                          video_id TEXT NOT NULL,
65                          start_time REAL NOT NULL,
66                          end_time REAL NOT NULL,
67                          duration REAL NOT NULL,
68                          server TEXT,
69                          receiver TEXT,
70                          metadata TEXT,
71                          FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72                      )
73                  """)
74
75                  # Events Table
76                  cursor.execute("""
```

```
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")

                if version < 2:
                    # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
                    # /api/process request. `status` is the EXECUTION state of the job
                    # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
                    # that failed validation) lives inside `result` JSON as
result["status"].
                    # `result` is the full sync-era response dict (incl. report paths), so
the
                    # observability report is the job's artifact, per the plan.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS jobs (
                            id TEXT PRIMARY KEY,
                            video_path TEXT NOT NULL,
```

```
                            video_id TEXT,
                            segmenter TEXT,
                            player_pool TEXT,
                            max_frames INTEGER,
                            status TEXT NOT NULL DEFAULT 'queued',
                            progress TEXT,
                            error TEXT,
                            result TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            started_at TIMESTAMP,
                            finished_at TIMESTAMP
                        )
                    """)
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
                    cursor.execute("PRAGMA user_version = 2")

                if version < 3:
                    # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
                    # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
                    # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
                    # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
                    # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
                    cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
                    cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
                    cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
                    cursor.execute("PRAGMA user_version = 3")
                # future: if version < 4: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 4

                conn.commit()

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            """Register or update a video row WITHOUT touching its child segments.

            Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
            with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
            That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
            before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
            row in place; clearing segments is now an explicit, deliberate operation
            (clear_video_data / prune_segments).
            """
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
                        "ON CONFLICT(id) DO UPDATE SET "
                        "filepath=excluded.filepath, status=excluded.status, "
                        "validation_results=excluded.validation_results",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
```

```
187                 return False
188
189         def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
190             """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
191
192             Captured before a (re)segmentation so the run's new rows (id > watermark) can be
193             told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
194             """
195             with self._connect() as conn:
196                 cur = conn.cursor()
197                 p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
198                                 (video_id,)).fetchone()[0]
199                 c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
200                                 (video_id,)).fetchone()[0]
201                 return int(p), int(c)
202
203         def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
204                            play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
205                            cand_after: Optional[int] = None) -> None:
206             """Delete play/candidate segments by id range (events cascade from
play_segments).
207
208             Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
209             successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
210             and keep the OLD ones when the run failed/produced nothing (``*_after``).
211             """
212             with self._connect() as conn:
213                 cur = conn.cursor()
214                 if play_upto is not None:
215                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
216                 if play_after is not None:
217                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
218                 if cand_upto is not None:
219                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
220                 if cand_after is not None:
221                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
222                 conn.commit()
223
224         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
225             try:
226                 with self._connect() as conn:
227                     cursor = conn.cursor()
228                     cursor.execute(
229                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
230                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
231                     )
232                     conn.commit()
233                     return cursor.lastrowid
234             except Exception as e:
235                 logger.error(f"Failed to add play segment: {e}")
236                 return None
237
```

```
        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
```

```
292             query = "SELECT * FROM candidate_segments WHERE video_id = ?"
293             params: List[Any] = [video_id]
294 
295             if detector:
296                 query += " AND detector = ?"
297                 params.append(detector)
298             if only_unlabeled:
299                 query += " AND human_label IS NULL"
300 
301             query += " ORDER BY start_time"
302 
303             candidates = []
304             with self._connect() as conn:
305                 rows = conn.cursor().execute(query, params).fetchall()
306                 for row in rows:
307                     d = dict(row)
308                     d["stats"] = json.loads(d["stats"] or "{}")
309                     d["detection_params"] = json.loads(d["detection_params"] or "{}")
310                     candidates.append(d)
311             return candidates
312 
313         def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
314             with self._connect() as conn:
315                 row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
316                 if row:
317                     d = dict(row)
318                     d["validation_results"] = json.loads(d["validation_results"] or "[]")
319                     return d
320             return None
321 
322         def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
323             """
324             Dynamically queries play segments based on filters:
325             - duration_min: float
326             - server: string
327             - receiver: string
328             - event_type: string (e.g. smash, foul)
329             """
330             query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
331             params = [video_id]
332 
333             if filters.get("duration_min"):
334                 query += " AND r.duration >= ?"
335                 params.append(filters["duration_min"])
336 
337             if filters.get("server"):
338                 query += " AND r.server = ?"
339                 params.append(filters["server"])
340 
341             if filters.get("receiver"):
342                 query += " AND r.receiver = ?"
343                 params.append(filters["receiver"])
344 
345             if filters.get("event_type"):
346                 # Check if there is an event of a specific type inside this segment
347                 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
348                 params.append(filters["event_type"])
349 
350             segments_list = []
351             with self._connect() as conn:
352                 cursor = conn.cursor()
353                 rows = cursor.execute(query, params).fetchall()
```

```
354
355                 for row in rows:
356                     r_dict = dict(row)
357                     r_id = r_dict["id"]
358                     r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
359
360                     # Fetch associated events
361                     event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
362                     r_dict["events"] = []
363                     for er in event_rows:
364                         e_dict = dict(er)
365                         e_dict["details"] = json.loads(e_dict["details"] or "{}")
366                         r_dict["events"].append(e_dict)
367
368                     segments_list.append(r_dict)
369
370             return segments_list
371
372         def clear_video_data(self, video_id: str):
373             with self._connect() as conn:
374                 conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
375                 conn.commit()
376
377         # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
378         # Writers follow the repo convention (swallow + logger.error, return bool); the
379         # worker logs loudly on a failed transition so a job can't silently wedge.
380
381         def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
382                        player_pool: Optional[List[str]] = None,
383                        max_frames: Optional[int] = None,
384                        policy: Optional[Dict[str, Any]] = None) -> bool:
385             """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
386             (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
387             try:
388                 with self._connect() as conn:
389                     conn.cursor().execute(
390                         "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
391                         "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
392                         (job_id, video_path, segmenter,
393                          json.dumps(player_pool) if player_pool is not None else None,
394                          max_frames, json.dumps(policy) if policy is not None else None)
395                     )
396                     conn.commit()
397                 return True
398             except Exception as e:
399                 logger.error(f"Failed to create job {job_id}: {e}")
400                 return False
401
402         def mark_job_running(self, job_id: str) -> bool:
403             try:
404                 with self._connect() as conn:
405                     conn.cursor().execute(
406                         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
407                         "progress='starting' WHERE id = ?", (job_id,))
408                     conn.commit()
409                 return True
410             except Exception as e:
411                 logger.error(f"Failed to mark job {job_id} running: {e}")
412                 return False
413
414         def set_job_progress(self, job_id: str, progress: str) -> bool:
415             try:
```

```
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set progress for job {job_id}: {e}")
                return False

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set video_id for job {job_id}: {e}")
                return False

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='done', progress='finished', result=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                        (json.dumps(result), job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} done: {e}")
                return False

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='failed', error=?, "
                        "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to mark job {job_id} failed: {e}")
                return False

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

        # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
-------------------------
```

```
        def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
            """Attach/replace a job's JSON ExecutionPolicy."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET policy=? WHERE id = ?",
                        (json.dumps(policy) if policy is not None else None, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set policy for job {job_id}: {e}")
                return False

        def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
            """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
            worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
            computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(
                        "UPDATE jobs SET status='waiting', "
                        "next_poll_at=datetime('now', ?), "
                        "progress=COALESCE(?, progress) WHERE id = ?",
                        (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to set job {job_id} waiting: {e}")
                return False

        def claim_due_job(self) -> Optional[Dict[str, Any]]:
            """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
            flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
            double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
            first. Returns the claimed job dict, or None if nothing is runnable."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    row = cur.execute(
                        "SELECT id, status FROM jobs WHERE status='queued' "
                        "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
                        "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
                    if not row:
                        return None
                    cur.execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
                        "WHERE id=? AND status=?", (row["id"], row["status"]))
                    conn.commit()
                    if cur.rowcount != 1:
                        return None                     # lost the race to a concurrent
worker
                return self.get_job(row["id"])
            except Exception as e:
                logger.error(f"Failed to claim a due job: {e}")
                return None

```

```
532         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
533             """Crash recovery: any job still queued/running from a previous process is dead
534             (the executor is in-process). Mark failed so clients see a terminal state, not a
535             hang. Called once at startup. Returns the number of jobs swept."""
536             try:
537                 with self._connect() as conn:
538                     cur = conn.cursor()
539                     cur.execute(
540                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
541                         "WHERE status IN ('queued', 'running')", (reason,))
542                     conn.commit()
543                     return cur.rowcount
544             except Exception as e:
545                 logger.error(f"Failed to sweep stale jobs: {e}")
546                 return 0
547
```

8. assistant (2026-06-14T10:41:49.662Z)

Worktree is set up. Now let me read the classifier and the fusion_hybrid wiring point, plus the existing migration test pattern.

9. user (2026-06-14T10:41:50.621Z)

```
1       """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3       The offline learned segmenter (ADR-004) must run locally with no external API and
4       no heavy ML stack ? the project already depends on numpy, and sklearn is not
5       installed. This implements just enough: standardized features, balanced-class
6       gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7       model is a small, human-readable, version-controllable file.
8
9       It deliberately stays simple: the rally-window features are few and roughly
10      linearly separable (high player motion + sustained activity => real rally), so a
11      well-regularized linear model is a sensible, inspectable baseline. Swap in a
12      richer model later behind the same fit/predict_proba/save/load interface.
13      """
14
15      import json
16      from typing import List, Optional
17
18      import numpy as np
19
20
21      class LogisticRegression:
22          def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23              self.lr = lr
24              self.epochs = epochs
25              self.l2 = l2
26              self.w: Optional[np.ndarray] = None
27              self.b: float = 0.0
28              self.mean: Optional[np.ndarray] = None
29              self.std: Optional[np.ndarray] = None
30              self.feature_names: Optional[List[str]] = None
31
32          @staticmethod
33          def _sigmoid(z: np.ndarray) -> np.ndarray:
34              # Clip for numerical stability (avoid overflow in exp).
35              return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37          def _standardize(self, X: np.ndarray) -> np.ndarray:
38              return (X - self.mean) / self.std
39
40          def fit(self, X, y, feature_names: Optional[List[str]] = None,
```

```
                class_weight: str = "balanced") -> "LogisticRegression":
            """Fit with optional balanced class weighting (rally windows are imbalanced
            ? many spurious candidates per real rally)."""
            X = np.asarray(X, dtype=float)
            y = np.asarray(y, dtype=float)
            if X.ndim != 2 or X.shape[0] == 0:
                raise ValueError("X must be a non-empty 2D array")
            self.feature_names = feature_names

            self.mean = X.mean(axis=0)
            self.std = X.std(axis=0)
            self.std[self.std == 0] = 1.0  # guard constant features
            Xs = self._standardize(X)
            n, d = Xs.shape

            if class_weight == "balanced":
                n_pos = max(1, int(y.sum()))
                n_neg = max(1, int((1 - y).sum()))
                w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
            else:
                w_pos = w_neg = 1.0
            sw = np.where(y == 1, w_pos, w_neg)

            self.w = np.zeros(d)
            self.b = 0.0
            for _ in range(self.epochs):
                p = self._sigmoid(Xs @ self.w + self.b)
                err = (p - y) * sw
                grad_w = Xs.T @ err / n + self.l2 * self.w
                grad_b = float(err.mean())
                self.w -= self.lr * grad_w
                self.b -= self.lr * grad_b
            return self

        def predict_proba(self, X) -> np.ndarray:
            if self.w is None:
                raise RuntimeError("model is not fitted")
            Xs = self._standardize(np.asarray(X, dtype=float))
            return self._sigmoid(Xs @ self.w + self.b)

        def predict(self, X, threshold: float = 0.5) -> np.ndarray:
            return (self.predict_proba(X) >= threshold).astype(int)

        # --- persistence (small JSON, no pickle/joblib) ---------------------------

        def to_dict(self) -> dict:
            return {
                "model": "logreg",
                "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
                "w": self.w.tolist() if self.w is not None else None,
                "b": self.b,
                "mean": self.mean.tolist() if self.mean is not None else None,
                "std": self.std.tolist() if self.std is not None else None,
                "feature_names": self.feature_names,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "LogisticRegression":
            m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
            m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
            m.b = float(d.get("b", 0.0))
            m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
            m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
            m.feature_names = d.get("feature_names")
```

```
            return m

        def save(self, path: str) -> None:
            with open(path, "w") as f:
                json.dump(self.to_dict(), f, indent=2)

        @classmethod
        def load(cls, path: str) -> "LogisticRegression":
            with open(path) as f:
                return cls.from_dict(json.load(f))

```

10. user (2026-06-14T10:41:50.855Z)

```
      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
      shuttle-seeded window via two overridable seams the engine exposes:

      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
        GPU-free, computed from the trajectory we already have); and, only when the person cue
        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
        **Persisted candidates remain the full superset** regardless, so the offline
experiment
        harness can re-score any variant.

      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
      experiment harness (EXPERIMENT_HARNESS.md), never silently.
      """
      from typing import Any, Dict, List, Optional, Tuple

      from backend.pipeline.segmenters.base import segmenter_registry
      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
      from backend.pipeline.detectors.base import DetectorRunner
      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
      from backend.pipeline.detectors import rally_gate
      from backend.pipeline.detectors import fusion_features as ff


      class FusionSegmenter(TrajectoryHybridSegmenter):
          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
```

```

        family = "fusion"
        display_label = "Fusion"

        def __init__(self, db: Any, config: Any):
            super().__init__(db, config)
            # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
            self._person: Optional[Any] = None
            # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
            # over the whole trajectory once per candidate window (it depends only on
`points`).
            self._axis_cache_key: Optional[int] = None
            self._axis_cache: Optional[tuple] = None

        @property
        def name(self) -> str:
            return "fusion_hybrid"

        @property
        def description(self) -> str:
            return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
                    "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
                    "AI provider sets semantic boundaries.")

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
            if substrate == "tracknet":
                cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
                return TrackNetRunner(cfg), {
                    "weights_path": cfg.weights_path,
                    "queue_length": cfg.queue_length,
                    "fusion_substrate": "tracknet",
                }
            wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
            return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}

        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
            # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
            # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
            # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
            gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                            min_crossings=0,
estimate=self._axis_estimate(points))
            stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

            fcfg = idx_cfg.get("fusion", {})
            if fcfg.get("cue_flags", {}).get("person", False):
                stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
            return stats

        def _axis_estimate(self, points: List[Any]) -> tuple:
```

```
93              """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94              computed once per video, not once per candidate window (it depends only on
points)."""
95              key = id(points)
96              if self._axis_cache_key != key or self._axis_cache is None:
97                  self._axis_cache_key = key
98                  self._axis_cache = rally_gate.estimate_play_axis(points)
99              est = self._axis_cache
100             assert est is not None
101             return est
102
103         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                              frame_width: float, video_path: str,
105                              fcfg: Dict[str, Any]) -> Dict[str, float]:
106             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107             frame range and compute the scale/translation-invariant person features. Lazily
builds
108             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110             from backend.pipeline.detectors.person_detector import PersonDetector
111             if self._person is None:
112                 self._person = PersonDetector(self.config)
113
114             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116             # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117             start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119             fw = det["frame_width"] or frame_width
120             fh = det["frame_height"]
121             # If we can't trust the frame dims, every normalised feature would silently
collapse
122             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123             if fw <= 0 or fh <= 0:
124                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126             poly = fcfg.get("court_polygon")
127             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                               court_polygon=polygon, net=net)
132
133         def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                                 window_stats: List[Dict[str, Any]],
135                                 idx_cfg: Dict[str, Any]) -> List[int]:
136             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137             (0) keep every window ? identical to the substrate. Persisted candidates are
138             untouched (full superset)."""
139             fcfg = idx_cfg.get("fusion", {})
140             min_crossings = int(fcfg.get("min_crossings", 0))
141             min_players = int(fcfg.get("min_players", 0))
142             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144             keep: List[int] = []
145             for i, st in enumerate(window_stats):
146                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                     continue
```

```
148                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                     continue
150                 keep.append(i)
151             return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155
```

11. assistant (2026-06-14T10:41:56.675Z)

Now let me look at the existing migration test pattern and how config/fusion config is structured, plus the eval experiment Variant for context on fusion config shape.

12. user (2026-06-14T10:41:59.035Z)

```
tests/__init__.py
tests/test_ablation.py
tests/test_annotation_providers.py
tests/test_api_endpoints.py
tests/test_async_jobs.py
tests/test_audio_features.py
tests/test_automated_provider.py
tests/test_base.py
tests/test_calibrate_local.py
tests/test_calibrate_wasb.py
tests/test_calibration.py
tests/test_cleanup_caches.py
tests/test_compiler.py
tests/test_config.py
tests/test_config_wasb.py
tests/test_database.py
tests/test_detector_keying.py
tests/test_distill_local.py
tests/test_eval_annotations.py
tests/test_eval_batch.py
tests/test_eval_classifier.py
tests/test_eval_harness.py
tests/test_eval_metrics.py
tests/test_eval_partial_labels.py
tests/test_eval_regression.py
tests/test_eval_stats.py
tests/test_eval_training.py
tests/test_execution_policy.py
tests/test_experiment.py
tests/test_experiment_honest.py
tests/test_export_wasb_segments.py
tests/test_fusion_compare.py
tests/test_fusion_features.py
tests/test_fusion_golden_telemetry.py
tests/test_fusion_hybrid.py
tests/test_gemini.py
tests/test_gemini_refine.py
tests/test_geometry.py
tests/test_gt_loader.py
tests/test_hashing.py
```

13. user (2026-06-14T10:41:59.902Z)

```
Found 12 files
C:\Users\avidu\Projects\bhi-servingbridge\tests\test_database.py
C:\Users\avidu\Projects\bhi-servingbridge\tests\test_async_jobs.py
C:\Users\avidu\Projects\bhi-servingbridge\docs\UI_ALPHA_EXECUTION_PLAN.md
```

C:\Users\avidu\Projects\bhi-servingbridge\docs\UI_PROPOSAL.md
C:\Users\avidu\Projects\bhi-servingbridge\docs\PLATFORM_ARCHITECTURE.md
C:\Users\avidu\Projects\bhi-servingbridge\docs\PERSONALIZATION_PLAN.md
C:\Users\avidu\Projects\bhi-servingbridge\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
C:\Users\avidu\Projects\bhi-servingbridge\docs\DEFERRED_HARDENING_PLAN.md
C:\Users\avidu\Projects\bhi-servingbridge\docs\CODE_MAP.md
C:\Users\avidu\Projects\bhi-servingbridge\docs\ANALYZERS_AND_RECONCILERS.md
C:\Users\avidu\Projects\bhi-servingbridge\backend\infrastructure\database.py
C:\Users\avidu\Projects\bhi-servingbridge\CLAUDE.md

14. user (2026-06-14T10:42:03.837Z)

```
1       import sqlite3
2       from backend.infrastructure.database import Database
3
4       def test_database_init(tmp_path):
5           db_path = tmp_path / "test.db"
6           Database(str(db_path))  # instantiation creates the schema
7
8           conn = sqlite3.connect(str(db_path))
9           cursor = conn.cursor()
10          cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
11          tables = [r[0] for r in cursor.fetchall()]
12          assert "videos" in tables
13          assert "play_segments" in tables
14          conn.close()
15
16      def test_database_video(tmp_path):
17          db = Database(str(tmp_path / "test.db"))
18
19          vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val": 1}])
20          assert vid_added is True
21
22          video = db.get_video("vid1")
23          assert video["id"] == "vid1"
24          assert video["filepath"] == "vid1.mp4"
25          assert video["status"] == "pending"
26          assert video["validation_results"] == [{"val": 1}]
27
28          db.clear_video_data("vid1")
29          assert db.get_video("vid1") is None
30
31      def test_database_play_segment(tmp_path):
32          db = Database(str(tmp_path / "test.db"))
33          db.add_video("vid1", "vid1.mp4", "pending", [])
34
35          sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA", "PlayerB", {"foo": "bar"})
36          assert sid is not None
37
38          db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
39
40          segments = db.query_play_segments("vid1", {"server": "PlayerA"})
41          assert len(segments) == 1
42          assert segments[0]["id"] == sid
43          assert segments[0]["start_time"] == 10.0
44          assert segments[0]["server"] == "PlayerA"
45          assert segments[0]["metadata"]["foo"] == "bar"
46
47          assert len(segments[0]["events"]) == 1
48          assert segments[0]["events"][0]["type"] == "smash"
49          assert segments[0]["events"][0]["details"]["speed"] == 300
50
51
52      # --- candidate_segments schema + behavior ----------------------------------
53
```

```
    # The schema contract: any future change to candidate_segments columns must
    # consciously update this set, otherwise the test below fails (regression guard).
    EXPECTED_CANDIDATE_COLUMNS = {
        "id", "video_id", "detector", "chunk_index", "start_time", "end_time",
        "duration", "confidence", "stats", "detection_params",
        "confirmed_segment_id", "human_label", "notes", "created_at",
    }


    def test_candidate_schema_contract(tmp_path):
        Database(str(tmp_path / "test.db"))  # instantiation creates the schema
        conn = sqlite3.connect(str(tmp_path / "test.db"))
        cols = conn.execute("PRAGMA table_info(candidate_segments)").fetchall()
        conn.close()

        names = {c[1] for c in cols}
        assert names == EXPECTED_CANDIDATE_COLUMNS

        # NOT NULL / PK flags on the load-bearing columns (cid, name, type, notnull, dflt,
pk)
        by_name = {c[1]: c for c in cols}
        assert by_name["id"][5] == 1  # primary key
        for col in ("video_id", "detector", "start_time", "end_time", "duration"):
            assert by_name[col][3] == 1, f"{col} should be NOT NULL"


    def test_schema_version_is_current(tmp_path):
        db_path = tmp_path / "test.db"
        Database(str(db_path))
        conn = sqlite3.connect(str(db_path))
        version = conn.execute("PRAGMA user_version").fetchone()[0]
        conn.close()
        # v3 = self-scheduling execution policy (EXECUTION_POLICY.md P2). Bump with each
migration.
        assert version == 3


    def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
        db_path = tmp_path / "legacy.db"
        # Simulate a pre-v1 database that only has the old tables and no version.
        conn = sqlite3.connect(str(db_path))
        conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
        conn.commit()
        conn.close()

        # Opening with the new Database should add candidate_segments without error...
        Database(str(db_path))
        # ...and running init again must be a no-op (idempotent).
        Database(str(db_path))

        conn = sqlite3.connect(str(db_path))
        tables = [r[0] for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
        version = conn.execute("PRAGMA user_version").fetchone()[0]
        conn.close()
        assert "candidate_segments" in tables
        assert "jobs" in tables  # v2 (async job model, #16)
        assert version == 3      # v3 = self-scheduling execution policy
(EXECUTION_POLICY.md P2)


    def test_candidate_round_trip(tmp_path):
        db = Database(str(tmp_path / "test.db"))
        db.add_video("vid1", "vid1.mp4", "processed", [])
```

```
115
116         cid = db.add_candidate_segment(
117             "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,
118             stats={"peak_velocity": 0.8, "track_id_count": 2},
119             detection_params={"frame_skip": 3},
120         )
121         assert cid is not None
122
123         cands = db.query_candidate_segments("vid1")
124         assert len(cands) == 1
125         c = cands[0]
126         assert c["detector"] == "yolo_hybrid"
127         assert c["start_time"] == 10.0
128         assert c["duration"] == 15.0
129         assert c["confidence"] == 0.42
130         assert c["stats"]["peak_velocity"] == 0.8
131         assert c["detection_params"]["frame_skip"] == 3
132         assert c["confirmed_segment_id"] is None
133
134         # detector filter
135         assert db.query_candidate_segments("vid1", detector="motion") == []
136         assert len(db.query_candidate_segments("vid1", detector="yolo_hybrid")) == 1
137
138
139     def test_candidate_link_and_unlabeled_filter(tmp_path):
140         db = Database(str(tmp_path / "test.db"))
141         db.add_video("vid1", "vid1.mp4", "processed", [])
142
143         cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
144         sid = db.add_play_segment("vid1", 1.0, 4.0)
145
146         assert db.link_candidate_to_segment(cid, sid) is True
147         assert db.query_candidate_segments("vid1")[0]["confirmed_segment_id"] == sid
148
149         # only_unlabeled returns it while human_label is NULL
150         assert len(db.query_candidate_segments("vid1", only_unlabeled=True)) == 1
151         with db._get_connection() as conn:
152             conn.execute("UPDATE candidate_segments SET human_label = ? WHERE id = ?",
153                          ("true_rally", cid))
154             conn.commit()
155         assert db.query_candidate_segments("vid1", only_unlabeled=True) == []
156
157
158     def test_candidate_cascade_on_video_delete(tmp_path):
159         db = Database(str(tmp_path / "test.db"))
160         db.add_video("vid1", "vid1.mp4", "processed", [])
161         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
162
163         db.clear_video_data("vid1")
164         assert db.query_candidate_segments("vid1") == []
165
```

15. assistant (2026-06-14T10:42:09.424Z)

Now let me look at the experiment.py Variant / FusionConfig shape and the per_user_gate to understand the existing data model for fusion_config and promotion evidence.

16. user (2026-06-14T10:42:10.363Z)

```
1       """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3       The owner adopted personalization as a **flag-gated** feature on a **generalization-
first**
4       baseline: a cue/variant B may be flipped ON *for one user* only when that user's own
5       held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
```

```
we
6       fall back to the shipped baseline (variant A). This module is the decision logic for
that
7       flip; it is pure (no I/O) and **reuses** ``backend.eval.eval_stats`` for every statistic
8       (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.
9
10      The counter-case this guards against (and its mitigations, implemented here):
11
12      1. **Minimum-N floor.** Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13         (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from
14         too few held-out videos are noise, not signal.
15      2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI
16         **excludes 0** *and* the paired sign test agrees (majority of videos win, p <=
threshold).
17         This is the global promotion bar applied per user.
18      3. **Structural guards are never disabled on "no lift".** A *structural* guard (e.g.
Gate-A
19         net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20         modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21         that simply lacks those failure cases must NEVER turn the guard OFF. For
``structural=True``
22         the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24      Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25      default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26      threaded through). Nothing here changes existing eval behaviour; callers opt in.
27      """
28      from __future__ import annotations
29
30      from dataclasses import dataclass, field
31      from typing import Any, Dict, Sequence
32
33      from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35      __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38      @dataclass
39      class PromotionDecision:
40          """Structured result of a per-user promotion decision.
41
42          - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
43            cue that cleared the full bar).
44          - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
45            always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
46            ``promote``.
47          - ``reason`` ? short human-readable explanation of the decision.
48          - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
49            so a caller can log/audit exactly why a user was or wasn't personalized.
50          """
51
52          promote: bool
53          keep_on: bool
54          reason: str
55          n: int
```

```
56          mean_delta: float
57          ci_low: float
58          ci_high: float
59          ci_excludes_zero: bool
60          sign_test: Dict[str, float] = field(default_factory=dict)
61          floor_met: bool = False
62          structural_protected: bool = False
63
64          def as_dict(self) -> Dict[str, Any]:
65              """Plain-dict view (for JSON logging / telemetry)."""
66              return {
67                  "promote": self.promote,
68                  "keep_on": self.keep_on,
69                  "reason": self.reason,
70                  "n": self.n,
71                  "mean_delta": self.mean_delta,
72                  "ci_low": self.ci_low,
73                  "ci_high": self.ci_high,
74                  "ci_excludes_zero": self.ci_excludes_zero,
75                  "sign_test": self.sign_test,
76                  "floor_met": self.floor_met,
77                  "structural_protected": self.structural_protected,
78              }
79
80
81      def should_promote_for_user(
82          a_scores: Sequence[float],
83          b_scores: Sequence[float],
84          *,
85          structural: bool = False,
86          min_n: int = 5,
87          p_threshold: float = 0.05,
88          confidence: float = 0.95,
89          n_boot: int = 2000,
90          seed: int = 0,
91          eps: float = 0.0,
92      ) -> PromotionDecision:
93          """Decide whether to flip variant B ON for ONE user from that user's held-out
scores.
94
95          ``a_scores`` / ``b_scores`` are the user's **per-video** held-out F1 (or any score)
for
96          variant A (control / shipped baseline) and variant B (with the cue), aligned by
video
97          (same order). The decision rules (faithful to the module docstring):
98
99          1. ``structural=True`` ? the guard is **never disabled on "no lift"**: return
100             ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
value is
101             in failure modes the user's corpus may not yet contain.
102         2. Otherwise promote **iff ALL** hold ? the minimum-N floor (``n >= min_n``), the
paired
103             ?F1 bootstrap CI **excludes 0**, and the paired sign test agrees (majority of
videos
104             win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
105
106         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
``paired_sign_test``);
107         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
108         """
109         n = min(len(a_scores), len(b_scores))
110         floor_met = n >= min_n
111
112         # --- Evidence (reuse eval_stats; never reinvent the statistics)
---------------------
```

```
        if n == 0:
            # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
            sign_test: Dict[str, float] = paired_sign_test([], eps)
            mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
        else:
            delta = paired_delta_ci(
                a_scores, b_scores,
                confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
            )
            mean_delta = float(delta["mean_delta"])
            ci_low = float(delta["ci_low"])
            ci_high = float(delta["ci_high"])
            ci_excludes_zero = bool(delta["ci_excludes_zero"])
            sign_test = delta["sign_test"]

        sign_p = float(sign_test.get("p_value", 1.0))
        sign_wins = float(sign_test.get("wins", 0.0))
        sign_losses = float(sign_test.get("losses", 0.0))
        # The sign test "agrees" with a promotion only if it is significant AND the majority
        # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
        # never satisfy the promotion bar.
        sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)

        # --- Rule 1: structural guards are never disabled on "no lift"
-----------------------
        if structural:
            return PromotionDecision(
                promote=False,           # nothing to "promote" ? it ships ON by design
                keep_on=True,            # ALWAYS kept ON, regardless of measured per-user
lift
                reason=(
                    "structural guard kept ON regardless of measured per-user lift "
                    "(value is in failure modes this user's corpus may not yet contain)"
                ),
                n=n,
                mean_delta=mean_delta,
                ci_low=ci_low,
                ci_high=ci_high,
                ci_excludes_zero=ci_excludes_zero,
                sign_test=sign_test,
                floor_met=floor_met,
                structural_protected=True,
            )

        # --- Rule 2: minimum-N floor (refuse before even looking at lift)
--------------------
        if not floor_met:
            return PromotionDecision(
                promote=False,
                keep_on=False,           # fall back to the shipped baseline (variant A)
                reason=(
                    f"below minimum-N floor (n={n} < min_n={min_n}); "
                    "falling back to shipped baseline"
                ),
                n=n,
                mean_delta=mean_delta,
                ci_low=ci_low,
                ci_high=ci_high,
                ci_excludes_zero=ci_excludes_zero,
                sign_test=sign_test,
                floor_met=False,
                structural_protected=False,
            )

```

```
174         # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
----------------
175         if ci_excludes_zero and sign_agrees:
176             promote = True
177             reason = (
178                 f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179                 f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180                 f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
181             )
182         else:
183             promote = False
184             misses = []
185             if not ci_excludes_zero:
186                 misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
187             if not sign_agrees:
188                 misses.append(
189                     f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}, "
190                     f"p={sign_p:.4f})"
191                 )
192             reason = (
193                 "insufficient evidence (" + "; ".join(misses) + "); "
194                 "falling back to shipped baseline"
195             )
196 
197         return PromotionDecision(
198             promote=promote,
199             keep_on=promote,             # non-structural: keep_on tracks promote exactly
200             reason=reason,
201             n=n,
202             mean_delta=mean_delta,
203             ci_low=ci_low,
204             ci_high=ci_high,
205             ci_excludes_zero=ci_excludes_zero,
206             sign_test=sign_test,
207             floor_met=True,
208             structural_protected=False,
209         )
210 
```

17. user (2026-06-14T10:42:11.277Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2 
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8 
9         - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
10        - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
11        - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
12        - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
13        - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
14        - `backend.eval.calibration`    ? `pct_improvement`
15 
16      The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17      (per-frame signals ? run once on the GPU/WSL box, persisted into
18      `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19      signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20      given persisted candidate features it produces `List[Interval]` deterministically,
21      with no GPU and zero production risk. So most experiments are pure offline
22      re-scoring of stored data and never touch the served pipeline.
23 
```

```
    Three modes (`EXPERIMENT_HARNESS.md` §5):
      - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
        aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
        held-out loop, but variant-parameterised.
      - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
        variants agree / diverge (A-only, B-only, agreed). The divergences are what a
        human spot-checks (and what two highlight reels would show side by side).
      - the promotion gate stays `run_regression.py` + `regression_baseline.json`
        (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

    Pure + offline + unit-tested. The only DB-touching helper is
    `load_video_candidates`, which reads persisted candidates via
    `Database.query_candidate_segments`.
    """
    from __future__ import annotations

    import json
    import logging
    import os
    from dataclasses import dataclass, field
    from datetime import datetime, timezone
    from hashlib import sha1
    from typing import Any, Callable, Dict, List, Optional, Sequence

    from backend.eval.calibration import pct_improvement
    from backend.eval.classifier import LogisticRegression
    from backend.eval.gemini_refine import merge_overlaps
    from backend.eval.metrics import Interval, match_intervals, score
    from backend.eval.regression import gt_hash
    from backend.eval.training import label_candidates
    from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
    from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
    from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

    logger = logging.getLogger(__name__)

    # Where a comparison run appends one reproducible record. Tracked location (NOT under
    # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
    # regression.DEFAULT_BASELINE_PATH.
    DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
    _METRICS = ("precision", "recall", "f1", "mean_iou")


    class ExperimentError(ValueError):
        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
        score an unlabeled video with no pinned model, or a gate referencing an absent
        feature)."""


    # ---------------------------------------------------------------------- Variant


    @dataclass
    class Variant:
        """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

        A variant turns one video's persisted candidate windows + features into rally
        intervals. Every knob defaults to the **control** behaviour (no gate, no learned
        filter), so a variant that merely *carries* a new, disabled cue is byte-identical
        to control ? the core no-regression guarantee.

        Fields:
          - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
```

```
87              for the leaderboard and for selecting the served path (the existing
88              `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89              New cues are recorded here default-OFF.
90            - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91              ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92              `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93              windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94            - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95              whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96              cue persists that feature).
97            - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98              `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99              required for `compare_unlabeled` on a learned variant.
100           - ``feature_names`` ? the persisted features the learned filter consumes. When set
101             WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102             (honest) ? the recipe, not a pinned artifact.
103           - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104             overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105         """
106
107         name: str
108         segmenter: str = "wasb_hybrid"
109         config_overrides: Dict[str, Any] = field(default_factory=dict)
110         cue_flags: Dict[str, bool] = field(default_factory=dict)
111         min_crossings: int = 0
112         min_players: int = 0
113         classifier_model: Optional[str] = None
114         feature_names: Optional[List[str]] = None
115         threshold: float = 0.5
116         merge_gap: float = 1.0
117         # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118         # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119         # failure where a fixed 0.5 zeroed out a small-candidate video.
120         tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121         calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first
122
123         def is_learned(self) -> bool:
124             """True if this variant applies a learned classifier (pinned model or
recipe)."""
125             return self.classifier_model is not None or bool(self.feature_names)
126
127         def to_dict(self) -> dict:
128             return {
129                 "name": self.name,
130                 "segmenter": self.segmenter,
131                 "config_overrides": self.config_overrides,
132                 "cue_flags": self.cue_flags,
133                 "min_crossings": self.min_crossings,
134                 "min_players": self.min_players,
135                 "classifier_model": self.classifier_model,
136                 "feature_names": self.feature_names,
137                 "threshold": self.threshold,
138                 "merge_gap": self.merge_gap,
139                 "tune_threshold": self.tune_threshold,
140                 "calibrate": self.calibrate,
141             }
142
143         @classmethod
144         def from_dict(cls, d: dict) -> "Variant":
145             known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
146             return cls(**{k: v for k, v in d.items() if k in known})
```

```
147
148         def spec_hash(self) -> str:
149             """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150             and makes a result reproducible. The pinned **control** variant always hashes
the
151             same, so a regression is always traceable to a recipe change, never to drift."""
152             blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153             return sha1(blob.encode()).hexdigest()[:12]
154
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ---------------------------------------------------------------- persisted candidates
162
163
164     @dataclass
165     class VideoCandidates:
166         """One video's persisted detection, ready for offline re-scoring.
167
168         ``intervals`` are the candidate windows; ``features`` is column-major
169         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171         with `load_video_candidates`, or directly in tests.
172         """
173
174         name: str
175         intervals: List[Interval]
176         features: Dict[str, List[float]] = field(default_factory=dict)
177         gts: Optional[List[Interval]] = None
178
179
180     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182         `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183         col = vc.features.get(name)
184         n = len(vc.intervals)
185         if col is None:
186             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                            name, vc.name)
188             return [0.0] * n
189         if len(col) != n:
190             raise ExperimentError(
191                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192         return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196         cols = [_feature_column(vc, name) for name in feature_names]
197         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
202         n = len(vc.intervals)
203         mask = [True] * n
204         if variant.min_crossings > 0:
205             col = _feature_column(vc, "net_crossings")
206             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
207         if variant.min_players > 0:
208             col = _feature_column(vc, "players_in_court")
```

```
            mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
        return mask


    def predict(variant: Variant, vc: VideoCandidates,
                model: Optional[LogisticRegression] = None,
                threshold: Optional[float] = None,
                calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
        """Variant prediction: gate by persisted features, optionally filter by a learned
        model, then merge. Pure and deterministic.

        ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
        fold; if omitted and the variant pins ``classifier_model``, that frozen model is
        loaded. A learned variant with neither a supplied nor a pinned model raises ? there
        is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
        fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
        """
        mask = _gate_mask(variant, vc)
        if variant.feature_names:
            if model is None:
                if variant.classifier_model is None:
                    raise ExperimentError(
                        f"variant {variant.name!r} is learned but has no model to predict "
                        f"with (pass a fitted model or set classifier_model)")
                model = LogisticRegression.load(variant.classifier_model)
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                      # grid 0.05..0.95 on the train
fold's rally F1
```

```
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
            report "how the shipped artifact does here", not for the promotion decision.
          - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
            train on the OTHER videos, predict the held-out one. The number to trust.
          - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
        """
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")

        per_video: List[Dict[str, Any]] = []
        predictions: Dict[str, List[Interval]] = {}

        recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
        pinned_model = (LogisticRegression.load(variant.classifier_model)
                        if variant.classifier_model is not None else None)
        all_model = None
        if recipe_lovo and not honest:
            all_model = _train(variant, videos, iou)

```

```
        for i, vc in enumerate(videos):
            cal: Optional[Callable[[float], float]] = None
            thr: Optional[float] = None
            if recipe_lovo and honest:
                others = [v for j, v in enumerate(videos) if j != i]
                if not others:
                    raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
                model: Optional[LogisticRegression] = _train(variant, others, iou)
                if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
                    assert model is not None                       # _train never returns
None
                    cal, thr = _calibrate_and_tune(variant, model, others, iou)
            elif recipe_lovo:                          # honest=False ? one model over all
videos
                model = all_model
            else:                                      # static variant or pinned model
                model = pinned_model
            preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
            assert vc.gts is not None                  # guarded at function top
            sc = score(preds, vc.gts, iou)
            per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
            predictions[vc.name] = preds

        aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
                     for m in _METRICS}
        return {
            "variant": variant.name,
            "spec_hash": variant.spec_hash(),
            "is_learned": variant.is_learned(),
            "honest_lovo": recipe_lovo and honest,
            "per_video": per_video,
            "aggregate": aggregate,
            "predictions": predictions,
        }


    def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
        """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
```

```
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out


    def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
        """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).

        ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
        so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
        the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
        """
        a_f1 = [p["f1"] for p in eval_a["per_video"]]
        b_f1 = [p["f1"] for p in eval_b["per_video"]]
        return {
            "variant_a_ci": _ci_block(eval_a),
            "variant_b_ci": _ci_block(eval_b),
            "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
            "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                             "variant_b": _segmentation_block(videos, eval_b)},
        }


    def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
        """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
        golden corpus and reports the **B ? A** deltas, per video and in aggregate.

        The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
        learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
        `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.

        ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
        paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
        existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
        shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
        bare LOVO mean at n?6 is not trustworthy on its own.
        """
        if len(videos) < 1:
            raise ExperimentError("compare needs at least one labeled video")
        eval_a = evaluate_variant(videos, variant_a, iou, honest)
        eval_b = evaluate_variant(videos, variant_b, iou, honest)

        delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
        delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}

        by_video_a = {p["video"]: p for p in eval_a["per_video"]}
        per_video_delta = [
            {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
            for p in eval_b["per_video"]
        ]
```

```
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436         result: Dict[str, Any] = {
437             "iou": iou,
438             "label_set_hash": gt_hash(all_gts),
439             "num_videos": len(videos),
440             "variant_a": eval_a,
441             "variant_b": eval_b,
442             "delta": delta,
443             "delta_pct": delta_pct,
444             "per_video_delta": per_video_delta,
445         }
446         if honest_stats:
447             result["honest"] = honest_summary(videos, eval_a, eval_b)
448         return result
449
450
451     # ------------------------------------------------------------- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                           agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462           - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
463           - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
464           - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
465             the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
a
468         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
469         `metrics.match_intervals` ? no new matching logic.
470         """
471         preds_a = predict(variant_a, video)
472         preds_b = predict(variant_b, video)
473         mr = match_intervals(preds_a, preds_b, agree_iou)
474
475         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
476                   for m in mr.matches]
477         agree_ious = [m.iou for m in mr.matches]
478         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
479         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
480
481         def _dur(ivs: List[Interval]) -> float:
482             return sum(e - s for s, e in ivs)
483
484         return {
485             "video": video.name,
486             "agree_iou": agree_iou,
487             "variant_a": variant_a.name,
488             "variant_b": variant_b.name,
489             "num_a": len(preds_a),
490             "num_b": len(preds_b),
491             "num_agreed": len(agreed),
492             "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
```

```
            "duration_a": _dur(preds_a),
            "duration_b": _dur(preds_b),
            "agreed": agreed,
            "a_only": a_only,
            "b_only": b_only,
        }


    # ---------------------------------------------------------------------- leaderboard / DB


    def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                          timestamp: Optional[str] = None) -> Dict[str, Any]:
        """Append a compact, reproducible record of a `compare()` result to the JSON
        leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
        `regression_baseline.json`; deliberately the size of a baseline file, not a service
        (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
        """
        row: Dict[str, Any] = {
            "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
            "iou": result.get("iou"),
            "label_set_hash": result.get("label_set_hash"),
            "num_videos": result.get("num_videos"),
            "variant_a": {"name": result["variant_a"]["variant"],
                          "spec_hash": result["variant_a"]["spec_hash"],
                          "aggregate": result["variant_a"]["aggregate"]},
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
        logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                    row["variant_a"]["name"], row["variant_b"]["name"],
                    (result.get("delta") or {}).get("f1", 0.0), path)
        return row


    def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                              gts: Optional[List[Interval]] = None,
                              feature_names: Sequence[str] = ()) -> VideoCandidates:
        """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.

        ``feature_names`` selects which persisted features to lift out of each row's
``stats``
        JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
        function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
        but works against any `Database` (including an in-memory one in tests).
        """
```

```
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562
```

18. assistant (2026-06-14T10:42:19.545Z)

I now have a full understanding. The `Variant` is the served-cfg shape (cue_flags, min_crossings, min_players, classifier_model, threshold, etc.). The fusion config in production is `idx_cfg["fusion"]` (a plain dict with `min_crossings`, `min_players`, `cue_flags`, etc.).

Let me check the per_user_eval.py module for the feature schema hash convention and how a "feature_schema_hash" is computed, plus confirm config typing and how to compute a hash. Let me also check how `run_all_tests.py` runs and whether ruff/mypy configs exist.

19. user (2026-06-14T10:42:20.978Z)

```
1       """Held-out-user LEARNING-CURVE eval ? "<K videos to superior quality" as a release
gate.
2
3       The personalization feature's promise is operational, not aspirational: *after the user
4       contributes K of their own videos, the personalized (candidate) recipe should beat the
5       generic (control) one on the user's STILL-UNSEEN footage.* This module turns that
promise
6       into a falsifiable criterion by sweeping the corpus size ``k`` and asking, at each
``k``,
7       **does the candidate beat control on a held-out split, honestly?**
8
9       It is the per-user analogue of `fusion_compare`: pure-CPU, no new scoring/stats math.
Every
10      fold reuses the shipped experiment primitives ?
11
12        - `experiment.predict`  ? variant ? held-out intervals (static variants need no fit);
13        - `experiment._train`   ? fit a learned variant on the corpus split (the same honest
14          train-on-others step `evaluate_variant`'s LOVO uses, here train-on-the-k-corpus);
15        - `metrics.score`       ? per-video held-out F1;
16        - `eval_stats.paired_delta_ci` ? the honest paired ?F1 (bootstrap CI + exact sign
test, C1).
17
18      **Split semantics (held-out USER split, NOT leave-one-out).** At each ``k`` the first
``k``
19      videos are the *personalization corpus* (fit/operating-point source) and the remaining
20      ``len(videos) - k`` are *held-out* (scored, never seen during fit). This mirrors the
product
21      reality ? the user has uploaded ``k`` videos and we want quality on their next one ? and
keeps
22      ``k`` and the evaluation set cleanly disjoint, so a measured lift can't be in-sample.
23
24      The headline ``smallest_k_with_lift`` is the smallest ``k`` at which the candidate beats
25      control with the **CI clearing 0 AND the sign test agreeing** (B wins on more held-out
videos
26      than it loses) ? the same promotion-grade bar `compare()`'s honest block uses, never a
bare
27      mean. ``None`` means the criterion was never met on this corpus (the honest negative).
28
```

```
    Pure, offline, deterministic given ``seed``; no GPU/torch/cv2; no file I/O (callers pass
    `VideoCandidates`). See `docs/EXPERIMENT_HARNESS.md` +
`docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
    """
    from __future__ import annotations

    from typing import Any, Dict, List, Optional, Sequence

    from backend.eval import eval_stats as _stats
    from backend.eval import experiment
    from backend.eval.classifier import LogisticRegression
    from backend.eval.experiment import ExperimentError, Variant, VideoCandidates
    from backend.eval.metrics import score


    def _fit_model(variant: Variant, corpus: Sequence[VideoCandidates],
                   iou: float) -> Optional[LogisticRegression]:
        """The fitted model a variant needs to predict held-out videos, or None if it needs
none.

        - **static variant** (no learned filter): None ? `predict` scores directly, no
leakage.
        - **pinned model** (``classifier_model`` set): the shipped model, loaded as-is (no
per-fold
          training ? reports "how the shipped artifact does on this user's held-out
footage").
        - **learned recipe** (``feature_names``, no pinned model): fit on the ``corpus``
split via
          the harness's own honest training step. The corpus and held-out sets are disjoint,
so the
          held-out scores are genuinely out-of-sample.
        """
        if variant.classifier_model is not None:
            return LogisticRegression.load(variant.classifier_model)
        if variant.feature_names:
            return experiment._train(variant, corpus, iou)
        return None


    def _heldout_f1s(variant: Variant, corpus: Sequence[VideoCandidates],
                     heldout: Sequence[VideoCandidates], iou: float) -> List[float]:
        """Per-held-out-video F1 for ``variant``, fit on ``corpus`` (if learned), scored on
        ``heldout``. Held-out videos are video-aligned with the returned list (caller pairs
B?A)."""
        model = _fit_model(variant, corpus, iou)
        f1s: List[float] = []
        for vc in heldout:
            assert vc.gts is not None  # guarded in learning_curve before any scoring
            preds = experiment.predict(variant, vc, model=model)
            f1s.append(score(preds, vc.gts, iou).f1)
        return f1s


    def learning_curve(videos: Sequence[VideoCandidates],
                       control_variant: Variant,
                       candidate_variant: Variant,
                       *,
                       iou: float = 0.5,
                       min_k: int = 2,
                       seed: int = 0) -> Dict[str, Any]:
        """Sweep corpus size ``k`` and report where the candidate first honestly beats
control.

        For one user's ``videos``, for each ``k`` in ``min_k .. len(videos) - 1``: fit/score
both
```

```
        variants on the first ``k`` videos (the personalization corpus) and evaluate on the
remaining
        held-out videos. Per-held-out-video F1s are paired B?A and summarised with
        ``eval_stats.paired_delta_ci`` (mean ?F1 + bootstrap CI + exact sign test).

        Returns ``{n_videos, min_k, iou, per_k, smallest_k_with_lift}`` where each ``per_k``
record is::

            {k, n_heldout, control_f1, candidate_f1,
             mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test}

        ``control_f1`` / ``candidate_f1`` are the mean held-out F1 of each variant at that
``k``.
        ``smallest_k_with_lift`` is the smallest ``k`` whose record shows a real lift ?
candidate
        above control, the ?F1 CI clearing 0, AND the sign test agreeing (B wins on strictly
more
        held-out videos than it loses) ? else ``None`` (the honest "never wins" answer).

        Pure + deterministic given ``seed`` (the only randomness is the CI bootstrap).
Raises
        `ExperimentError` on unlabeled videos or a too-small corpus (no valid ``k`` to
sweep).
        """
        n = len(videos)
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(
                    f"{vc.name} has no golden labels; learning_curve needs labeled held-out
videos")
        if min_k < 1:
            raise ExperimentError(f"min_k must be >= 1 (got {min_k})")
        if n <= min_k:
            # Every k in [min_k, n-1] needs k >= min_k corpus videos AND >= 1 held-out
video, so
            # n must exceed min_k. Too-few videos ? no sweepable k; report an empty, honest
curve.
            return {
                "n_videos": n,
                "min_k": min_k,
                "iou": iou,
                "per_k": [],
                "smallest_k_with_lift": None,
            }

        per_k: List[Dict[str, Any]] = []
        smallest_k_with_lift: Optional[int] = None
        for k in range(min_k, n):
            corpus = videos[:k]
            heldout = videos[k:]
            control_f1s = _heldout_f1s(control_variant, corpus, heldout, iou)
            candidate_f1s = _heldout_f1s(candidate_variant, corpus, heldout, iou)

            paired = _stats.paired_delta_ci(control_f1s, candidate_f1s, seed=seed)
            sign_test = paired["sign_test"]
            # Honest lift = the candidate is above control AND the CI clears 0 AND the sign
test
            # agrees (B wins on strictly more held-out videos than it loses) ? the promotion
bar.
            control_mean = sum(control_f1s) / len(control_f1s) if control_f1s else 0.0
            candidate_mean = sum(candidate_f1s) / len(candidate_f1s) if candidate_f1s else
0.0
            has_lift = (
                candidate_mean > control_mean
                and bool(paired["ci_excludes_zero"])
```

```
                and paired["mean_delta"] > 0.0
                and sign_test["wins"] > sign_test["losses"]
            )
            per_k.append({
                "k": k,
                "n_heldout": len(heldout),
                "control_f1": control_mean,
                "candidate_f1": candidate_mean,
                "mean_delta": paired["mean_delta"],
                "ci_low": paired["ci_low"],
                "ci_high": paired["ci_high"],
                "ci_excludes_zero": bool(paired["ci_excludes_zero"]),
                "sign_test": sign_test,
            })
            if has_lift and smallest_k_with_lift is None:
                smallest_k_with_lift = k

        return {
            "n_videos": n,
            "min_k": min_k,
            "iou": iou,
            "per_k": per_k,
            "smallest_k_with_lift": smallest_k_with_lift,
        }

```

20. user (2026-06-14T10:42:22.533Z)

```
mypy.ini
mypy.ini
pytest.ini
ruff.toml
---PYPROJECT---
```

21. assistant (2026-06-14T10:42:28.317Z)

```
No pyproject. Let me check the ruff/mypy/pytest configs and how tests are run.
```

22. user (2026-06-14T10:42:30.404Z)

```
===RUFF===
```

Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would bury blame history for zero behavior value).

```
#
```

Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle warnings (W) + bugbear (B). Deferred with intent: I (import-sort) and UP (pyupgrade) ? both are large mechanical diffs; adopt via a dedicated PR.

```
target-version = "py38"

```

scratch/ = throwaway debugging scripts that may hit real APIs; it is also excluded from pytest collection (see pytest.ini).

```
extend-exclude = ["scratch"]

[lint]
extend-select = ["W", "B"]

[lint.per-file-ignores]
```

Entrypoint scripts deliberately run setup (load_dotenv, logging config,

sys.path) before importing backend modules ? import order is load-bearing.

```
"backend/main.py" = ["E402"]
"scripts/run_phase3_eval.py" = ["E402"]
"scripts/run_process_and_stitch.py" = ["E402"]
===MYPY===
[mypy]
```

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade. Ratchet: when you touch a quarantined module, try removing its entry.

```
ignore_missing_imports = True
```

backend/ is a namespace package (no root __init__.py) ? required so mypy maps files to backend.* module names instead of duplicating top-level names.

```
explicit_package_bases = True
```

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set only in main()); properly fixed by the deferred app-factory/lifespan refactor (BACKLOG).

```
[mypy-backend.main]
ignore_errors = True
```

Legacy demo segmenter (fabricated metadata; coverage-omitted too).

```
[mypy-backend.pipeline.segmenters.motion]
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...] annotation) ? ratchet those out in a focused typing pass, not here. (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```
[mypy-backend.pipeline.segmenters.yolo_hybrid]
ignore_errors = True

[mypy-backend.pipeline.segmenters.gemini]
ignore_errors = True
```

Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.

```
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True
===PYTEST===
[pytest]
```

When `pytest` is invoked without an explicit path, only collect from tests/. scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.) and should never run as part of the regular test suite.

```
testpaths = tests
```

23. user (2026-06-14T10:42:33.060Z)

```
ruff 0.15.16
mypy 2.1.0 (compiled: yes)
```

```
pytest 9.0.3
numpy 2.4.6
```

24. assistant (2026-06-14T10:42:37.668Z)

```
All tools available, including numpy. Let me check how mypy is invoked in CI (which paths) and
the config package structure so my new module fits the typing gate.
```

25. user (2026-06-14T10:42:40.178Z)

```
ci.yml
---
  # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
  # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
    name: Ruff lint
      - name: Install ruff
        run: python -m pip install ruff
      - name: Ruff check
        run: ruff check . --output-format github
  # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
  # up. Typed core deps are installed so mypy sees their real signatures
    name: Mypy (lenient baseline)
      - name: Install mypy + typed core deps
        run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-genai
numpy
      - name: Mypy
        run: mypy backend training
        run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-genai
        run: python -m pytest tests/test_import_smoke.py -v
        run: python -m pip install -r requirements.txt pytest pytest-cov httpx  # httpx: fastapi
TestClient
      # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
        run: python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-
under=70
```

26. user (2026-06-14T10:42:41.146Z)

```
Exit code 2
audio
detectors
segmenters
stitcher
validators
---personalization?---
ls: cannot access 'backend/pipeline/personalization': No such file or directory
```

27. assistant (2026-06-14T10:42:47.969Z)

```
CI runs `mypy backend training`, `ruff check .`, and `run_all_tests.py` with a 70% coverage
floor. Let me check the config package and how `IndexingConfig` / fusion config is shaped to
know what `served_cfg` type to expect, then look at the `__init__.py` pattern for a package.
```

28. user (2026-06-14T10:42:49.575Z)

```
backend/pipeline/detectors/__init__.py
backend/pipeline/segmenters/__init__.py
---config---
__init__.py
loader.py
models.py
```

29. user (2026-06-14T10:42:51.004Z)

```
C:\Users\avidu\Projects\bhi-servingbridge\backend\config\models.py:79:    """Config for the
```

```
multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
C:\Users\avidu\Projects\bhi-servingbridge\backend\config\models.py:86:    is ``fusion_hybrid``
(config.json ``indexing.default_segmenter``); the offline experiment
C:\Users\avidu\Projects\bhi-servingbridge\backend\config\models.py:89:    **Reversible ?
rollback is a single flag:** set ``indexing.fusion.min_crossings: 0`` to
C:\Users\avidu\Projects\bhi-servingbridge\backend\config\models.py:91:    to revert the served
segmenter entirely. With ``min_crossings=0`` the fusion segmenter
C:\Users\avidu\Projects\bhi-servingbridge\backend\config\models.py:94:    The person cue (Gate
B) stays OFF by default: the fusion segmenter persists the shuttle
C:\Users\avidu\Projects\bhi-servingbridge\backend\config\models.py:104:    # Windowing velocity
threshold for the fusion family (the engine reads
C:\Users\avidu\Projects\bhi-servingbridge\backend\config\models.py:106:    # equal to
indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
C:\Users\avidu\Projects\bhi-servingbridge\backend\config\models.py:159:    fusion: FusionConfig
= Field(default_factory=FusionConfig)
```

30. user (2026-06-14T10:42:55.468Z)

```
75          min_free_gb: float = 0.0
76
77
78      class FusionConfig(BaseModel):
79          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81          **Gate A (the held-shuttle guard) is ON by default** (``min_crossings=2``): a window
82          must show the shuttle crossing the net >= 2 times to reach the AI handoff, dropping
the
83          held-shuttle / one-sided-feed / warm-up false positives. This is the promoted,
validated
84          operating point (?F1 +0.074, 6/6 golden, sign-test p=0.031, CI [+0.042,+0.105];
over-seg
85          1.68?1.01 ? see docs/QUALITY_ITERATIONS.md). It only takes effect when the served
segmenter
86          is ``fusion_hybrid`` (config.json ``indexing.default_segmenter``); the offline
experiment
87          harness is unaffected (it sets ``Variant.min_crossings`` explicitly, never reading
this).
88
89          **Reversible ? rollback is a single flag:** set ``indexing.fusion.min_crossings: 0``
to
90          restore the pre-Gate-A (ungated) handoff, or ``indexing.default_segmenter:
wasb_hybrid``
91          to revert the served segmenter entirely. With ``min_crossings=0`` the fusion
segmenter
92          reproduces the substrate (wasb_hybrid) byte-for-byte.
93
94          The person cue (Gate B) stays OFF by default: the fusion segmenter persists the
shuttle
95          net-crossing count always, and ? only when ``cue_flags['person']`` is true ? the
person-cue
96          features; ``min_players`` (Gate B) defaults to 0 = OFF. The court mask is optional:
97          ``court_polygon=None`` ? Gate B counts every detection (player-count-only); supplied
?
98          off-court persons (spectators / adjacent courts) are excluded. **The persisted
candidate
99          set is the full superset regardless of any gate** ? gating only filters the AI
handoff, so
100         the offline harness can re-score any variant.
101         """
102         # Which trajectory substrate seeds the windows (shuttle stays primary).
103         substrate: Literal["wasb", "tracknet"] = "wasb"
104         # Windowing velocity threshold for the fusion family (the engine reads
105         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
```

```
107         velocity_threshold: float = 0.02
108         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109         # enabled ? so the default path never imports torch / touches the GPU.
110         cue_flags: dict[str, bool] = Field(default_factory=dict)
111         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112         # Default 2 = the PROMOTED held-shuttle guard (a real rally crosses the net >=2x; a
single
113         # service feed / held shuttle crosses ~once). Set 0 to turn the gate OFF (ungated
handoff;
114         # reproduces the substrate exactly). Persisted candidates are ALWAYS the full
superset for
115         # offline re-scoring, regardless of this threshold.
116         min_crossings: int = Field(default=2, ge=0)
117         # Served Gate B: when the person cue is on, drop windows with median in-court
players
118         # < this. 0 = OFF.
119         min_players: int = Field(default=0, ge=0)
120         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
121         court_polygon: Optional[list[list[float]]] = None
122         # Net line for the person two-sided-motion split (person features only ? the shuttle
123         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
124         net_axis: Literal["x", "y"] = "y"
125         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
126
127
128     class IndexingConfig(BaseModel):
129         default_segmenter: str = "yolo_hybrid"
130         use_gpu: bool = True
131         motion_threshold: float = 0.08
132         min_segment_duration: float = 3.0
133         dead_time_merge_gap: float = 2.0
134         chunk_duration: float = 90.0
135         chunk_overlap: float = 10.0
136         detector_model: str = "fasterrcnn_resnet50_fpn"
137         detector_score_threshold: float = 0.5
138         yolo_velocity_threshold: float = 0.15
139         yolo_frame_skip: int = 3
140         yolo_tracker: str = "bytetrack.yaml"
141         yolo_prefetch_workers: int = 2
142         min_action_window_duration: float = 2.0
143         log_yolo_candidates: bool = True
144         store_yolo_candidates: bool = True
145         ai_handoff_padding: float = 2.0
146         ai_max_workers: int = 5
147         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
148         # chunks of a high-bitrate (4K) source blow past the provider upload cap
149         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
150         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
151         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
152         # Matches gemini_refine's --clip-scale-width default.
153         ai_chunk_scale_width: int = 1280
154         # Optional debug knob: trim every video to the first N seconds before processing.
155         debug_duration: Optional[float] = None
156
157         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
158         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
159         fusion: FusionConfig = Field(default_factory=FusionConfig)
160
161         @field_validator("default_segmenter")
162         @classmethod
163         def segmenter_must_be_registered(cls, v: str) -> str:
164             # Registry check happens at runtime in main/segmenter selection.
```

31. assistant (2026-06-14T10:43:01.463Z)

I now have a complete picture. The served fusion config is a plain dict (`idx_cfg["fusion"]`) with keys: `substrate, velocity_threshold, cue_flags, min_crossings, min_players, court_polygon, net_axis, net_position`. The `serving.py` overlay resolver will overlay the active user_models row's `fusion_config` JSON onto this served dict.

Let me start the chapter and begin building. First the migrations.

32. user (2026-06-14T10:43:02.100Z)

Chapter marked: "Implementation". Continue your current work.

33. assistant (2026-06-14T10:43:04.702Z)

Now let me build the v4 and v5 migrations into database.py.

34. user (2026-06-14T10:43:28.634Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

35. assistant (2026-06-14T10:43:32.182Z)

The harness wants me to re-read in this worktree path (I read the main repo copy). Let me read the worktree's database.py to confirm content matches (it should ? same origin/master), then edit.

36. user (2026-06-14T10:43:33.331Z)

```
150                 if version < 3:
151                     # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152                     # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153                     # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154                     # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155                     # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156                     cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157                     cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158                     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159                     cursor.execute("PRAGMA user_version = 3")
160                 # future: if version < 4: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 4
161
162                 conn.commit()
163
164         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
```

37. assistant (2026-06-14T10:43:35.439Z)

Identical content. Now the edit will apply.

38. user (2026-06-14T10:43:52.911Z)

The file C:/Users/avidu/Projects/bhi-servingbridge/backend/infrastructure/database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

39. assistant (2026-06-14T10:44:02.564Z)

Now add the CRUD helpers (`upsert_user_model` / `get_active_user_model`) at the end of the Database class. Let me read the end of the file to place them after `fail_stale_jobs`.

40. user (2026-06-14T10:44:03.386Z)

```
595             except Exception as e:
596                 logger.error(f"Failed to claim a due job: {e}")
597                 return None
598 
599         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
600             """Crash recovery: any job still queued/running from a previous process is dead
601             (the executor is in-process). Mark failed so clients see a terminal state, not a
602             hang. Called once at startup. Returns the number of jobs swept."""
603             try:
604                 with self._connect() as conn:
605                     cur = conn.cursor()
606                     cur.execute(
607                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
608                         "WHERE status IN ('queued', 'running')", (reason,))
609                     conn.commit()
610                     return cur.rowcount
611             except Exception as e:
612                 logger.error(f"Failed to sweep stale jobs: {e}")
613                 return 0
614 
```

41. user (2026-06-14T10:44:24.348Z)

```
The file C:/Users/avidu/Projects/bhi-servingbridge/backend/infrastructure/database.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

42. assistant (2026-06-14T10:44:27.875Z)

Now create the personalization package and `serving.py`. First the package `__init__.py`.

43. assistant (2026-06-14T10:44:30.915Z)

```
"""Personalization serving groundwork (PERSONALIZATION_PLAN.md Phase 1).

This package holds the **serving bridge** between the per-user model store (DB v5
``user_models``) and the production decision seam. It is deliberately additive and
**not yet wired into the served pipeline**: resolving a per-user ``fusion_config`` /
classifier here changes nothing about what gets served until a FUTURE, owner-gated PR
wires it into ``fusion_hybrid._select_for_handoff``. See ``serving.py``.
"""
```

44. user (2026-06-14T10:44:30.943Z)

```
File created successfully at: C:/Users/avidu/Projects/bhi-
servingbridge/backend/pipeline/personalization/__init__.py (file state is current in your
context ? no need to Read it back)
```

45. assistant (2026-06-14T10:44:34.594Z)

Now `serving.py`. The overlay resolver: takes the served fusion cfg dict, the active user_models row's `fusion_config` JSON, falls back to baseline if no active model or `n_videos_at_fit < min_n`.

46. assistant (2026-06-14T10:44:58.462Z)

```
"""Per-user serving bridge ? resolve a user's fusion config + classifier from the v5 store.

PERSONALIZATION_PLAN.md §2 (Phase 1, the "#1 wall"): a personalized model is a per-user
``Variant`` (cue_flags + ``min_crossings`` / ``min_players`` + an optional
``LogisticRegression``
+ threshold). It does NOT require a segmenter rewrite ? it is a per-user ``FusionConfig``
```

```
overlay
resolved UPSTREAM of detection, plus a classifier the decision seam re-scores persisted features
with. This module is that resolution step, and **only** that step.

What this module does (additive, pure-ish ? one DB read, no writes):
  - ``resolve_user_fusion_config`` ? overlay the active ``user_models`` row's ``fusion_config``
    onto the served fusion config dict, returning the config the served path WOULD use. Falls
    back to the served baseline (a copy, unchanged) when there is no active model, or the model
    is below the promotion-confidence floor (``n_videos_at_fit < min_n``), or it carries no
    fusion overlay. The min-N floor here is the SAME ``per_user_gate`` promotion floor: below it
    a per-user A/B is not trusted enough to override a baseline default (PLAN §1.3).
  - ``load_user_classifier`` ? rehydrate the stored ``classifier_json`` into a
    ``LogisticRegression`` via ``from_dict`` (None when the user has no active model or it is a
    config-only model). ``LogisticRegression.load`` is never called in ``backend/pipeline/``
    today ? this is the load half of the serving bridge.

? NOT WIRED INTO THE SERVED PATH. ``fusion_hybrid._select_for_handoff`` still reads only the
static ``idx_cfg["fusion"]`` ? it does NOT call anything here. Wiring this resolver into that
seam (so a pinned, promoted per-user model actually changes the served handoff) is the NEXT,
**owner-gated** PR, because it is the first change to served behavior. Until then this is pure
groundwork: import it, unit-test it, but nothing in production reads it. With NULL ``user_id``
(the local install) and no active model, ``resolve_user_fusion_config`` returns the served
config unchanged ? byte-identical to today.
"""
from __future__ import annotations

import copy
from typing import Any, Dict, Optional

from backend.eval.classifier import LogisticRegression

__all__ = [
    "DEFAULT_MIN_N",
    "resolve_user_fusion_config",
    "load_user_classifier",
]

#: Promotion-confidence floor (PERSONALIZATION_PLAN.md §1.3, owner-confirmed ``min_n = 5``).
#: A per-user model fit on fewer than this many of the user's videos is NOT trusted enough to
#: override a baseline fusion default ? we fall back to the served baseline. This mirrors
#: ``backend.eval.per_user_gate.should_promote_for_user``'s ``min_n`` default.
DEFAULT_MIN_N = 5


def resolve_user_fusion_config(
    db: Any,
    user_id: Optional[str],
    served_cfg: Dict[str, Any],
    *,
    min_n: int = DEFAULT_MIN_N,
) -> Dict[str, Any]:
    """Resolve the fusion config the served path should use for ``user_id``.

    Overlays the active ``user_models`` row's ``fusion_config`` (a partial per-user
    ``FusionConfig`` dict ? e.g. raised ``min_crossings`` / ``min_players`` / flipped
    ``cue_flags``) onto a COPY of ``served_cfg`` (the baseline served fusion config dict, i.e.
    ``idx_cfg["fusion"]``). The original ``served_cfg`` is never mutated.

    Falls back to the served baseline (an unmodified deep copy) when ANY of:
      - the user has no active model;
      - the active model was fit below the promotion floor (``n_videos_at_fit < min_n``) ? too
        little of the user's own footage to trust a per-user override (PLAN §1.3);
      - the active model carries no ``fusion_config`` overlay (a classifier-only model).

    So a brand-new / NULL (local) user with no active model gets the served baseline unchanged
```

```
    ? never denied service, never altered. The overlay is a shallow key-level merge: per-user
    keys win, every key the user did not set keeps the baseline value.
    """
    base = copy.deepcopy(served_cfg)
    model = db.get_active_user_model(user_id)
    if not model:
        return base
    if int(model.get("n_videos_at_fit") or 0) < min_n:
        # Below the trust floor ? keep the baseline (the conservative default).
        return base
    overlay = model.get("fusion_config")
    if not overlay:
        return base  # config-only fallback: a classifier-only model has nothing to overlay
    base.update(overlay)
    return base


def load_user_classifier(db: Any, user_id: Optional[str]) -> Optional[LogisticRegression]:
    """Rehydrate the active per-user classifier for ``user_id``, or None.

    Returns a ``LogisticRegression`` rebuilt from the stored ``classifier_json`` (via
    ``from_dict``) when the user has an active model that carries one; None when there is no
    active model or it is a config-only (no-classifier) model. The caller (the FUTURE served
    wiring) would feed this model's ``predict_proba`` over the persisted ``window_stats`` ?
    the same math ``experiment.predict`` runs offline. Nothing calls this on the served path
    yet; it is the load half of the serving bridge.
    """
    model = db.get_active_user_model(user_id)
    if not model:
        return None
    blob = model.get("classifier_json")
    if not blob:
        return None
    return LogisticRegression.from_dict(blob)
```

47. user (2026-06-14T10:44:58.480Z)

File created successfully at: C:/Users/avidu/Projects/bhi-servingbridge/backend/pipeline/personalization/serving.py (file state is current in your context ? no need to Read it back)

48. assistant (2026-06-14T10:45:04.269Z)

Now let me update the existing `test_database.py` schema-version + idempotency assertions (they assert version == 3) and add the new candidate column to the contract set. Then write comprehensive new tests.

49. user (2026-06-14T10:45:07.074Z)

The file C:/Users/avidu/Projects/bhi-servingbridge/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

50. user (2026-06-14T10:45:13.280Z)

The file C:/Users/avidu/Projects/bhi-servingbridge/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

51. user (2026-06-14T10:45:20.019Z)

The file C:/Users/avidu/Projects/bhi-servingbridge/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

52. assistant (2026-06-14T10:45:22.863Z)

Now add comprehensive migration + back-compat + CRUD tests to `test_database.py`. Let me append at the end of the file.

53. user (2026-06-14T10:45:23.411Z)

```
158
159
160     def test_candidate_cascade_on_video_delete(tmp_path):
161         db = Database(str(tmp_path / "test.db"))
162         db.add_video("vid1", "vid1.mp4", "processed", [])
163         db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
164
165         db.clear_video_data("vid1")
166         assert db.query_candidate_segments("vid1") == []
167
```

54. user (2026-06-14T10:46:01.168Z)

The file C:/Users/avidu/Projects/bhi-servingbridge/tests/test_database.py has been updated successfully. (file state is current in your context ? no need to Read it back)

55. assistant (2026-06-14T10:46:04.865Z)

Now I need to verify that SQLite supports `ALTER TABLE ... DROP COLUMN` (used in my migration test). That requires SQLite 3.35+. Let me check the SQLite version, and also confirm `upsert_user_model`'s rowid SELECT works. Let me run the database tests now.

56. user (2026-06-14T10:46:07.460Z)

```
sqlite 3.50.4
...................                                                  [100%]
19 passed in 0.79s
```

57. assistant (2026-06-14T10:46:09.947Z)

All 19 database tests pass (SQLite 3.50.4 supports DROP COLUMN). Now write the serving.py tests.

58. assistant (2026-06-14T10:46:33.454Z)

```
"""Tests for the per-user serving bridge (backend/pipeline/personalization/serving.py).

Phase-1 groundwork: resolve a per-user fusion-config overlay + rehydrate a stored classifier
from the v5 ``user_models`` store. These exercise the resolver against a REAL (in-memory)
``Database`` so the round trip (upsert ? resolve) is end-to-end. Nothing here touches the
served pipeline ? the resolver is not wired into ``_select_for_handoff`` yet.
"""
import numpy as np
import pytest

from backend.eval.classifier import LogisticRegression
from backend.infrastructure.database import Database
from backend.pipeline.personalization import serving

# A representative served fusion config dict (mirrors idx_cfg["fusion"] from FusionConf
BASELINE_FUSION = {
    "substrate": "wasb",
    "velocity_threshold": 0.02,
    "cue_flags": {},
    "min_crossings": 2,
    "min_players": 0,
    "court_polygon": None,
    "net_axis": "y",
```

```
    "net_position": 0.5,
}


@pytest.fixture
def db(tmp_path):
    return Database(str(tmp_path / "serving.db"))
```

--- resolve_user_fusion_config ----------

```
def test_no_active_model_returns_baseline_unchanged(db):
    out = serving.resolve_user_fusion_config(db, "user-1", BASELINE_FUSION)
    assert out == BASELINE_FUSION


def test_null_user_no_model_is_byte_identical_baseline(db):
    """The local install (NULL user_id) with no active model gets the served config
unchanged."""
    out = serving.resolve_user_fusion_config(db, None, BASELINE_FUSION)
    assert out == BASELINE_FUSION


def test_resolve_does_not_mutate_input(db):
    db.upsert_user_model("user-1", version=1, fusion_config={"min_crossings": 9},
                         n_videos_at_fit=5, is_active=True)
    snapshot = dict(BASELINE_FUSION)
    serving.resolve_user_fusion_config(db, "user-1", BASELINE_FUSION)
    assert BASELINE_FUSION == snapshot  # the caller's served_cfg is never mutated


def test_active_model_overlays_fusion_keys(db):
    db.upsert_user_model(
        "user-1", version=1,
        fusion_config={"min_crossings": 4, "cue_flags": {"person": True}, "min_players": 2},
        n_videos_at_fit=6, is_active=True,
    )
    out = serving.resolve_user_fusion_config(db, "user-1", BASELINE_FUSION)
    # Overlaid keys win...
    assert out["min_crossings"] == 4
    assert out["cue_flags"] == {"person": True}
    assert out["min_players"] == 2
    # ...untouched keys keep the baseline value.
    assert out["substrate"] == "wasb"
    assert out["net_position"] == 0.5


def test_below_min_n_falls_back_to_baseline(db):
    """A model fit below the promotion floor (n_videos_at_fit < min_n) is not trusted to
override."""
    db.upsert_user_model("user-1", version=1, fusion_config={"min_crossings": 99},
                         n_videos_at_fit=serving.DEFAULT_MIN_N - 1, is_active=True)
    out = serving.resolve_user_fusion_config(db, "user-1", BASELINE_FUSION)
    assert out == BASELINE_FUSION  # overlay refused, baseline kept


def test_at_min_n_boundary_applies_overlay(db):
    db.upsert_user_model("user-1", version=1, fusion_config={"min_crossings": 7},
                         n_videos_at_fit=serving.DEFAULT_MIN_N, is_active=True)
    out = serving.resolve_user_fusion_config(db, "user-1", BASELINE_FUSION)
    assert out["min_crossings"] == 7


def test_custom_min_n_threshold(db):
```

```
    db.upsert_user_model("user-1", version=1, fusion_config={"min_crossings": 7},
                         n_videos_at_fit=3, is_active=True)
    # Default floor (5) refuses; an explicit lower floor accepts.
    assert serving.resolve_user_fusion_config(db, "user-1", BASELINE_FUSION) == BASELINE_FUSION
    out = serving.resolve_user_fusion_config(db, "user-1", BASELINE_FUSION, min_n=3)
    assert out["min_crossings"] == 7


def test_classifier_only_model_keeps_baseline_config(db):
    """A model with a classifier but no fusion overlay leaves the served fusion config alone."""
    db.upsert_user_model(
        "user-1", version=1, fusion_config=None,
        classifier_json={"model": "logreg", "w": [0.1], "b": 0.0,
                         "mean": [0.0], "std": [1.0], "feature_names": ["net_crossings"]},
        n_videos_at_fit=6, is_active=True,
    )
    out = serving.resolve_user_fusion_config(db, "user-1", BASELINE_FUSION)
    assert out == BASELINE_FUSION
```

--- load_user_classifier ----------

```
def test_load_classifier_none_when_no_model(db):
    assert serving.load_user_classifier(db, "nobody") is None


def test_load_classifier_none_for_config_only_model(db):
    db.upsert_user_model("user-1", version=1, fusion_config={"min_crossings": 3},
                         n_videos_at_fit=6, is_active=True)
    assert serving.load_user_classifier(db, "user-1") is None


def test_load_classifier_round_trips_predictions(db):
    """A fitted model stored as classifier_json reloads to an equivalent LogisticRegression."""
    rng = np.random.default_rng(0)
    X = rng.normal(size=(40, 2))
    y = (X[:, 0] + X[:, 1] > 0).astype(int)
    trained = LogisticRegression(epochs=200).fit(X, y, feature_names=["a", "b"])

    db.upsert_user_model("user-1", version=1, classifier_json=trained.to_dict(),
                         n_videos_at_fit=6, is_active=True)
    loaded = serving.load_user_classifier(db, "user-1")
    assert loaded is not None
    assert loaded.feature_names == ["a", "b"]
    # Predictions match the original model bit-for-bit (same weights/standardisation).
    np.testing.assert_allclose(loaded.predict_proba(X), trained.predict_proba(X))


def test_null_user_classifier_round_trip(db):
    rng = np.random.default_rng(1)
    X = rng.normal(size=(20, 1))
    y = (X[:, 0] > 0).astype(int)
    trained = LogisticRegression(epochs=100).fit(X, y, feature_names=["x"])
    db.upsert_user_model(None, version=1, classifier_json=trained.to_dict(),
                         n_videos_at_fit=5, is_active=True)
    loaded = serving.load_user_classifier(db, None)
    assert loaded is not None
    np.testing.assert_allclose(loaded.predict_proba(X), trained.predict_proba(X))
```

59. user (2026-06-14T10:46:33.473Z)

File created successfully at: C:/Users/avidu/Projects/bhi-servingbridge/tests/test_personalization_serving.py (file state is current in your context ? no need to Read it back)

60. user (2026-06-14T10:46:38.919Z)

```
............                                                             [100%]
12 passed in 0.69s
```

61. assistant (2026-06-14T10:46:43.430Z)

All 12 serving tests pass. Now let me run ruff and mypy on the new/changed files, then the full suite.

62. user (2026-06-14T10:46:44.958Z)

```
All checks passed!
```

63. user (2026-06-14T10:47:18.519Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 96 source files
```

64. assistant (2026-06-14T10:47:21.542Z)

Ruff clean, mypy clean (only pre-existing notes, "Success: no issues found"). Now run the full test suite via `run_all_tests.py`.

65. user (2026-06-14T10:48:20.988Z)

```
                    )
                """)
                cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
                cursor.execute("PRAGMA user_version = 2")

            if version < 3:
                # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
                # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
                # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
                # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
                # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
                cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
                cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
                cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
                cursor.execute("PRAGMA user_version = 3")

            if version < 4:
                # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
                # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
                # path can scope rows to an owner. **NULL = local install = byte-identical to
                # today** ? every existing row stays NULL and every existing query (which never
                # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive.
                # Additive ONLY: no served-behavior change rides on this slice.
                cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
>               cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
E               sqlite3.OperationalError: no such table: candidate_segments

backend\infrastructure\database.py:169: OperationalError
========================== short test summary info ==========================
```

```
FAILED tests/test_async_jobs.py::test_migration_fresh_db_is_v3_with_jobs_table
FAILED tests/test_async_jobs.py::test_migration_upgrades_v1_db_in_place - sql...
2 failed, 629 passed, 2 skipped in 56.42s
=========================================================================
 RESULT: FAIL - pytest exit 1
=========================================================================
```

66. assistant (2026-06-14T10:48:24.588Z)

Two pre-existing tests in `test_async_jobs.py` simulate older DB states and need updating. One reveals a real ordering issue: my v4 migration assumes `candidate_segments` exists, but a legacy DB that jumps from a pre-v1 state created only `videos`. Let me look at those tests.

67. user (2026-06-14T10:48:25.929Z)

```
79:def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
80-    db = Database(str(tmp_path / "fresh.db"))
81-    with db._get_connection() as conn:
82-        assert conn.execute("PRAGMA user_version").fetchone()[0] == 3
83-        cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
84-    assert {"id", "video_path", "video_id", "status", "progress", "error", "result"} <= cols
85-    assert {"policy", "next_poll_at"} <= cols           # v3 self-scheduling columns
86-
87-
88:def test_migration_upgrades_v1_db_in_place(tmp_path):
89-    path = str(tmp_path / "v1.db")
90-    conn = sqlite3.connect(path)
91-    conn.execute(
92-        "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
93-        "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)")
94-    conn.execute("PRAGMA user_version = 1")
95-    conn.commit()
96-    conn.close()
97-
98-    db = Database(path)  # ladder should take it 1 ? 3
99-    with db._get_connection() as c:
100-        assert c.execute("PRAGMA user_version").fetchone()[0] == 3
101-        assert c.execute(
102-            "SELECT name FROM sqlite_master WHERE type='table' AND name='jobs'").fetchone()
103-
104-
105-# --- submit-time fast-fail -------------------------------------------------------
106-
107-def test_submit_returns_202_with_job_id(env):
108-    client, db, tmp_path, mp = env
109-    vid_file = _video(tmp_path)
110-    mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
111-    mp.setattr(m.segmenter_registry, "get", lambda name: _FakeSeg)
112-    r = client.post("\api\process", json={"video_path": str(vid_file), "segmenter": "x"})
113-    assert r.status_code == 202
114-    body = r.json()
115-    assert body["job_id"]
116-    assert body["status"] == "queued"
117-    assert body["poll"].endswith(body["job_id"])
118-
119-
120-def test_submit_rejects_null_byte_path(env):
121-    client = env[0]
122-    r = client.post("\api\process", json={"video_path": "a\x00b.mp4"})
123-    assert r.status_code == 400
124-
125-
126-def test_submit_missing_file_is_404(env):
127-    client, db, tmp_path, mp = env
128-    r = client.post("\api\process", json={"video_path": str(tmp_path / "nope.mp4")})
```

```
129-    assert r.status_code == 404
130-
131-
132-def test_submit_unknown_segmenter_400_and_no_job_row(env):
133-    client, db, tmp_path, mp = env
```

68. assistant (2026-06-14T10:48:40.248Z)

The first test (`test_migration_fresh_db_is_v3_with_jobs_table`) asserts version 3 on a fresh DB ? needs bumping to 5. The second (`test_migration_upgrades_v1_db_in_place`) creates a v1 DB with ONLY a `videos` table (no `candidate_segments`) and expects the ladder to take it to v3. This exposes a real bug: my v4 migration does `ALTER TABLE candidate_segments` but that legacy DB never had `candidate_segments`.

Wait ? looking at the v1 migration: it does `CREATE TABLE IF NOT EXISTS candidate_segments` only when `version < 1`. A DB pinned at `user_version = 1` (but missing `candidate_segments`) is an inconsistent simulated state ? the test sets version=1 without creating candidate_segments. In reality a true v1 DB *would* have candidate_segments. But my v4 ALTER would still fail on such a DB.

The robust fix: guard the v4 ALTERs so they're safe even if a column/table is missing. But the cleaner and more correct fix is to make the v4 migration resilient ? `candidate_segments` is guaranteed by the v1 block in a genuine ladder, but the simulated test is legitimately checking the ladder runs. Let me make the migration defensive by ensuring `candidate_segments` exists (the v1 CREATE uses `IF NOT EXISTS`, so it's safe to also not assume). The simplest correct approach: in the v4 block, only ALTER columns that don't already exist, and the test's v1 DB is genuinely malformed.

Let me check: the real issue is the test simulates a v1 DB missing `candidate_segments`. Since a real v1 DB always has it (created in the `version < 1` block), the test is simulating an impossible state. However, to be safe and not break the existing test contract, I'll make the v4 migration tolerant: ensure the column-add is guarded. Let me update both tests to reflect the new ladder end-state (v5), and make the v4 ALTER on candidate_segments resilient by guaranteeing the table exists first.

The cleanest fix that keeps the existing test meaningful: the v1 migration block already creates `candidate_segments` with `IF NOT EXISTS` ? but only runs when `version < 1`. For a DB at version=1 lacking the table, that block is skipped. I'll add an idempotent safety `CREATE TABLE IF NOT EXISTS candidate_segments` isn't ideal (duplicates schema). Better: update the two async-job tests since they assert the old terminal version, and fix the second test to include candidate_segments (matching a real v1 DB). Let me update both tests.

69. user (2026-06-14T10:48:52.638Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

70. assistant (2026-06-14T10:48:55.079Z)

Need to read the file first in this worktree.

71. user (2026-06-14T10:48:56.181Z)

```
79      def test_migration_fresh_db_is_v3_with_jobs_table(tmp_path):
80          db = Database(str(tmp_path / "fresh.db"))
81          with db._get_connection() as conn:
82              assert conn.execute("PRAGMA user_version").fetchone()[0] == 3
83              cols = {r["name"] for r in conn.execute("PRAGMA table_info(jobs)").fetchall()}
84          assert {"id", "video_path", "video_id", "status", "progress", "error", "result"} <=
cols
85          assert {"policy", "next_poll_at"} <= cols           # v3 self-scheduling columns
86
87
88      def test_migration_upgrades_v1_db_in_place(tmp_path):
89          path = str(tmp_path / "v1.db")
```

```
90          conn = sqlite3.connect(path)
91          conn.execute(
92              "CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT NOT NULL, "
93              "processed_at TIMESTAMP, status TEXT NOT NULL, validation_results TEXT)")
94          conn.execute("PRAGMA user_version = 1")
95          conn.commit()
96          conn.close()
97
98          db = Database(path)  # ladder should take it 1 ? 3
99          with db._get_connection() as c:
100             assert c.execute("PRAGMA user_version").fetchone()[0] == 3
101             assert c.execute(
102                 "SELECT name FROM sqlite_master WHERE type='table' AND
name='jobs'").fetchone()
103
```

72. user (2026-06-14T10:49:10.544Z)

```
The file C:/Users/avidu/Projects/bhi-servingbridge/tests/test_async_jobs.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

73. assistant (2026-06-14T10:49:20.212Z)

Now there's still the underlying robustness concern: a real v1/v2 DB always has `candidate_segments`, so the v4 ALTER is safe in practice. But to be defensive against any malformed legacy DB and to make the migration genuinely robust, let me check whether any other test or real-world path could hit a missing `candidate_segments` at v4. The v1 block always creates it, so any DB at version >= 1 has it. That's a safe invariant. Let me re-run the full suite.

74. user (2026-06-14T10:50:15.092Z)

```
.......................................................................... [ 68%]
.......................................................................... [ 79%]
.......................................................................... [ 90%]
.....................................................                    [100%]
631 passed, 2 skipped in 52.29s
============================================================================
 RESULT: PASS - all tests green
============================================================================
```

75. assistant (2026-06-14T10:50:18.498Z)

All 631 tests pass. Let me also confirm coverage stays above the 70% floor with the CI invocation, and re-run ruff/mypy to be certain everything is green after the test edits.

76. user (2026-06-14T10:50:22.811Z)

```
All checks passed!
===MYPY===
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 96 source files
```

77. user (2026-06-14T10:51:36.392Z)

```
backend\utils\run_telemetry.py                       126     12    90%
backend\utils\video.py                                25      3    88%
training\__init__.py                                   0      0   100%
training\gen0\__init__.py                              0      0   100%
training\gen0\harness.py                             108     11    90%
----------------------------------------------------------------------
```

```
TOTAL                                                6574   1247    81%
Required test coverage of 70% reached. Total coverage: 81.03%
631 passed, 2 skipped, 4 warnings in 71.01s (0:01:11)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

78. assistant (2026-06-14T10:51:40.100Z)

All green: ruff clean, mypy clean, 631 tests pass, coverage 81% (above the 70% floor). Let me check my new modules' specific coverage to confirm they're well-tested.

79. user (2026-06-14T10:51:44.420Z)

```
.............................C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\ast.py
:50: ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001D8FF6F6F20>
backend\infrastructure\database.py              318    138    57%   40-41, 252-254, 262-268,
279-289, 301-303, 314-316, 337-339, 351-353, 401-402, 409-410, 414-415, 454-467, 470-479,
482-490, 493-501, 506-516, 520-529, 533-541, 546-555, 561-572, 579-597, 603-613, 680-682
backend\pipeline\personalization\serving.py      26      0   100%
TOTAL                                            344    138    60%
31 passed in 1.53s
```

80. assistant (2026-06-14T10:51:48.324Z)

serving.py is at 100% coverage. The database.py uncovered lines are pre-existing job-store methods not exercised by these two test files (they're covered elsewhere ? the full suite shows 81%). My new CRUD methods (`upsert_user_model`/`get_active_user_model`) are fully exercised.

Now let me commit. First, re-verify the branch and that only my changes are staged (branch-safety protocol).

81. user (2026-06-14T10:51:50.045Z)

```
feat/personalization-store-and-bridge
---HEAD---
e75ec35 feat(eval): LOSO/LOGO ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
---STATUS---
 M backend/infrastructure/database.py
 M tests/test_async_jobs.py
 M tests/test_database.py
?? backend/pipeline/personalization/
?? tests/test_personalization_serving.py
---vs origin/master---
```

82. assistant (2026-06-14T10:51:54.767Z)

Branch is correct, HEAD is at origin/master (no sibling commits), and exactly my intended files are changed. Let me stage explicit paths and commit.

83. user (2026-06-14T10:51:56.030Z)

```
warning: in the working copy of 'backend/pipeline/personalization/__init__.py', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'backend/pipeline/personalization/serving.py', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_personalization_serving.py', LF will be replaced by
CRLF the next time Git touches it
M  backend/infrastructure/database.py
A  backend/pipeline/personalization/__init__.py
A  backend/pipeline/personalization/serving.py
M  tests/test_async_jobs.py
M  tests/test_database.py
A  tests/test_personalization_serving.py
```

84. assistant (2026-06-14T10:52:04.639Z)

All six intended files staged, nothing else. Now commit.

85. user (2026-06-14T10:52:15.339Z)

```
[feat/personalization-store-and-bridge 15dea32] feat(personalization): per-user store (v4/v5) +
serving bridge groundwork (Phase 1, additive)
 6 files changed, 616 insertions(+), 7 deletions(-)
 create mode 100644 backend/pipeline/personalization/__init__.py
 create mode 100644 backend/pipeline/personalization/serving.py
 create mode 100644 tests/test_personalization_serving.py
```

86. assistant (2026-06-14T10:52:18.787Z)

Committed. Let me verify the commit is the only one ahead of origin/master, then push and create the PR.

87. user (2026-06-14T10:52:23.807Z)

```
---only my commit?---
15dea32 feat(personalization): per-user store (v4/v5) + serving bridge groundwork (Phase 1,
additive)
---push---
remote:
remote: Create a pull request for 'feat/personalization-store-and-bridge' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-
indexer/pull/new/feat/personalization-store-and-bridge
remote:
branch 'feat/personalization-store-and-bridge' set up to track 'origin/feat/personalization-
store-and-bridge'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      feat/personalization-store-and-bridge -> feat/personalization-store-and-
bridge
```

88. assistant (2026-06-14T10:52:30.509Z)

Pushed (still exactly one commit ahead). Now create the PR.

89. user (2026-06-14T10:52:52.907Z)

https://github.com/avidullu/badminton-highlight-indexer/pull/150

90. assistant (2026-06-14T10:52:58.174Z)

PR #150 created. Now let me wait for CI to go green before self-merging. Let me check the CI status.

91. user (2026-06-14T10:53:00.386Z)

```
Full test suite (offline, mocked)       pending 0       https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27496543819/job/81271552462
Mypy (lenient baseline) pending 0       https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552470
Ruff lint       pending 0       https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552483
Syntax sweep + import smoke (no GPU stack)      pending 0
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27496543819/job/81271552484
```

92. assistant (2026-06-14T10:53:02.826Z)

CI is running. Let me wait for it to complete using a watch loop.

93. user (2026-06-14T10:55:07.896Z)

```
Refreshing checks status every 20 seconds. Press Ctrl+C to quit.

Mypy (lenient baseline) pass    30s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552470
Full test suite (offline, mocked)       pending 0       https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27496543819/job/81271552462
Ruff lint       pass    7s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552483
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27496543819/job/81271552484
Refreshing checks status every 20 seconds. Press Ctrl+C to quit.

Mypy (lenient baseline) pass    30s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552470
Full test suite (offline, mocked)       pending 0       https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27496543819/job/81271552462
Ruff lint       pass    7s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552483
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27496543819/job/81271552484
Full test suite (offline, mocked)       pass    2m1s    https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27496543819/job/81271552462
Mypy (lenient baseline) pass    30s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552470
Ruff lint       pass    7s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552483
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27496543819/job/81271552484
Full test suite (offline, mocked)       pass    2m1s    https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27496543819/job/81271552462
Mypy (lenient baseline) pass    30s     https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552470
Ruff lint       pass    7s      https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27496543819/job/81271552483
Syntax sweep + import smoke (no GPU stack)      pass    19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27496543819/job/81271552484
```

94. assistant (2026-06-14T10:55:11.447Z)

All four CI checks pass. This is an additive/safe good-engineering change within the merge agreement. Self-merging with squash + branch delete.

95. user (2026-06-14T10:55:17.560Z)

```
 backend/infrastructure/database.py           | 157 ++++++++++++++++++-
 backend/pipeline/personalization/__init__.py |   8 ++
 backend/pipeline/personalization/serving.py  | 103 +++++++++++++
 tests/test_async_jobs.py                     |  18 ++-
 tests/test_database.py                       | 193 ++++++++++++++++++++++-
 tests/test_personalization_serving.py        | 144 ++++++++++++++++++
 6 files changed, 616 insertions(+), 7 deletions(-)
 create mode 100644 backend/pipeline/personalization/__init__.py
 create mode 100644 backend/pipeline/personalization/serving.py
 create mode 100644 tests/test_personalization_serving.py
```

96. user (2026-06-14T10:55:22.895Z)


```
{"mergeCommit":"8439c24ab8e4a46a6b9c148d359cb5fb46108b2a","mergedAt":"2026-06-
14T10:55:15Z","state":"MERGED"}
```


97. assistant (2026-06-14T10:55:26.553Z)

Merged. Now clean up: remove the worktree, delete the local branch, prune. The worktree's cwd is the worktree itself, so I'll run cleanup from the main repo path.

98. user (2026-06-14T10:55:29.402Z)

```
Exit code 1
---prune---
---delete branch---
error: branch 'feat/personalization-store-and-bridge' not found
```

99. assistant (2026-06-14T10:55:33.495Z)

The worktree remove produced no output (success) but the branch -D errored. Let me check the actual state ? the branch may have been auto-deleted, or the worktree removal partially completed.

100. user (2026-06-14T10:55:35.104Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer                                f6acde5
[docs/served-gate-a-verification]
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy  5cb7bbf
[bhi-execpolicy]
C:/Users/avidu/Projects/bhi-evalserve                                              e75ec35
[feat/eval-serve-windowing-alignment]
C:/Users/avidu/Projects/bhi-revert                                                 7883a7c
[fix/revert-gate-a-default]
C:/Users/avidu/Projects/bhi-servedverify                                           fd0d7b3
[feat/served-gate-a-regression]
---branches---
+ bhi-execpolicy
* docs/served-gate-a-verification
+ feat/eval-serve-windowing-alignment
+ feat/served-gate-a-regression
+ fix/revert-gate-a-default
  master
---dir exists?---
GONE
```

101. assistant (2026-06-14T10:55:40.330Z)

The worktree directory is GONE and `bhi-servingbridge` no longer appears in `worktree list` ? the removal succeeded. The branch `feat/personalization-store-and-bridge` is also already gone from the local branch list (the worktree removal cleaned it up, which is why `branch -D` said "not found"). Everything is cleaned up correctly.

Note: there are several other active worktrees and branches here (sibling sessions' WIP) ? I did not touch any of them. My cleanup only removed my own worktree and branch. The prune already ran. Let me do a final confirmation that nothing of mine lingers.

102. user (2026-06-14T10:55:42.751Z)

```
exit: 1 (1 = no match = clean)
---master has the merge?---
8439c24 feat(personalization): per-user store (v4/v5) + serving bridge groundwork (Phase 1,
additive) (#150)
e75ec35 feat(eval): LOSO/LOGO ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
570cf2b feat(fusion): flip Gate-A (held-shuttle guard) to the served default ? config-reversible
(#146)
```

103. assistant (2026-06-14T10:56:05.677Z)

No `servingbridge` branch remains locally or remotely, the worktree is gone, and the squash-merge commit `8439c24` is on `origin/master`. Everything is clean.

Done ? Personalization Phase-1, first safe slice (serving-bridge groundwork)

PR #150 ? https://github.com/avidullu/badminton-highlight-indexer/pull/150 ? **MERGED** (squash commit `8439c24` on `origin/master`). Worktree removed, branch deleted, pruned.

What was built (additive; NULL user_id = local = byte-identical to today; zero served-behavior change)

- **DB v4** ? nullable `user_id` on `videos` + `candidate_segments` (+ partial indexes `WHERE user_id IS NOT NULL`). Existing writers never set it; existing readers never filter on it.
- **DB v5** ? `user_models` store per PLAN §2 (`user_id, version, baseline_version, feature_schema_hash, fusion_config, classifier_json, promotion_evidence, n_videos_at_fit, is_active`) with unique `(user_id, version)` + a partial-unique "one active model per user" index. CRUD: `upsert_user_model` (idempotent; activating a row demotes the prior active one; correct `IS NULL` local-install handling) / `get_active_user_model`.
- **`backend/pipeline/personalization/serving.py`** ? `resolve_user_fusion_config(db, user_id, served_cfg, *, min_n=5)` (overlay active model's `fusion_config` onto a copy of the served cfg; fall back to baseline when no active model / below the `min_n=5` promotion floor / no overlay; never mutates input) + `load_user_classifier` (`LogisticRegression.from_dict` on stored `classifier_json`). **NOT wired into `_select_for_handoff`** ? clear TODO/docstrings mark that as the next owner-gated PR.

Test results / gates

- ruff **clean** · `mypy backend training` **clean** · full `pytest` **631 passed, 2 skipped** · coverage **81%** (floor 70%); `serving.py` at **100%** line coverage. All four CI checks green before merge.
- Tests cover: migration version bump (3?5) + idempotency + v3?v5 upgrade preserving rows; NULL-user back-compat; `user_models` schema contract + CRUD (idempotent upsert, active demotion, NULL-local round trip); serving overlay (baseline fallback, min_n floor, no-mutation, custom floor, classifier `predict_proba` round-trip).
- One real ordering point handled: the v4 ALTER on `candidate_segments` requires the table to exist (it always does at version ? 1, created in the v1 block). Two pre-existing `test_async_jobs.py` migration tests were updated to the new v5 terminal version, and the simulated-v1 DB test now includes `candidate_segments` (as a genuine v1 DB has).

EXACT description of the NEXT PR (owner-gated ? first served-behavior change)

Title: `feat(personalization): wire the per-user serving bridge into fusion_hybrid._select_for_handoff (owner-gated)`

Scope: In `backend/pipeline/segmenters/fusion_hybrid.py::_select_for_handoff`, when a per-user model is pinned active AND the promotion gate says ON, replace the static `idx_cfg["fusion"]` read with the resolved per-user path:
1. Resolve the effective fusion config via `serving.resolve_user_fusion_config(db, user_id, idx_cfg["fusion"])` and apply the resolved `min_crossings` / `min_players` / `cue_flags` gates (instead of reading the raw served cfg).
2. If `serving.load_user_classifier(db, user_id)` returns a model, apply `model.predict_proba` over `window_stats` (the same math `experiment.predict()` runs offline ? reuse `experiment._feature_matrix`/the model's `feature_names`) and gate windows by the per-user `threshold`. **Persisted candidates remain the full superset** (only the handoff is gated ? unchanged invariant).
3. Plumb `user_id` into the segmenter's decision context (today it has none ? add an optional `user_id`, default None ? exactly today's behaviour; NULL/no-active-model resolves to the served baseline, so the default path is byte-identical).

Gating / safety: This is the first change to served behavior, so it is **owner-gated** and must ship flag-gated/default-OFF behind a golden-set regression run (per PLAN §1.4 / §5 and the no-regression gate). Pair it with the `min_n=5` promotion floor and Gate-A structural protection already in `backend/eval/per_user_gate.py`. Tests: a pinned active user model with a raised `min_crossings` / a learned classifier changes the handoff for that user only, while NULL/no-model users get the identical served output as before.

Out of scope for that PR (later Phase-1/2 items): the `/retune` + `/model` trust-feature endpoints, `set_user_id`/`set_human_label` writers, and the correction?label write path.